

DSA study plan

basic data structures

chapter 10

arrays

0	1	2	3	4	5
1	4	2	5	3	6

- the most basic data structure (used in pretty much every other data structure types) that stores items at contiguous locations.
- a **static** array is a fixed length container with n elements that are **indexable** from the range $[0, n-1]$.
- a **dynamic** array is not fixed in size, so elements can be inserted and removed.
 - to implement a dynamic array, you can use static arrays with an initial capacity. if adding a new element to the static array exceeds its capacity, then create a new static array from this array by doubling the size and copying the original elements into it.
 - the array in the above pic already reached its full capacity, so if a new element is added, the array will first be doubled in size and the original elements will be copied into it:
[1, 4, 2, 5, 3, 6, _, _, _, _, _]. the new element can then be inserted in the empty slot: [1, 4, 2, 5, 3, 6, 7, _, _, _, _].
- given elements of different size, a program will store **pointers** to the elements in an array
 - pointer is stored to the address of the element rather than the value.

- and because the size of the address is fixed and independent of the size of the element, the computer can access the pointers in constant time.

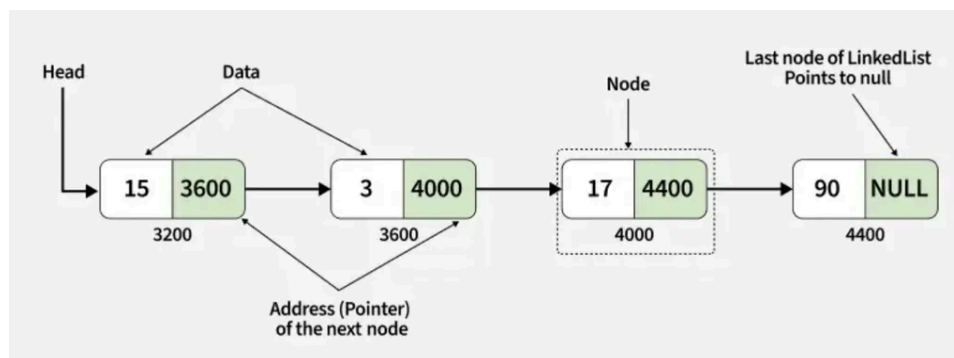
operations/time complexity:

- **traversal:** process of accessing each element in the array sequentially (usually by iterating through the array)
- **insertion:** insert an element at any given position in the array.
- **deletion:** delete an element at any given position in the array.

	static array	dynamic array
traversal	$O(n)$	$O(n)$
insertion	-	$O(n)$
deletion	-	$O(n)$

- note: insertion and deletion are linear because the entire array gets shifted when you insert or delete an element.
- note: since static arrays are fixed in size you cannot add or remove elements to change its size, hence the insertion and deletion operations are n/a on a static array. but you can replace already existing elements with new ones by referencing the index. so "A[2] = 4" for the above array in the pic means: replace the element in the array at index 2 with the new element 4.

linked lists



- a linked list is a (non contiguous) sequential list of nodes that hold data which point to other nodes also containing data.

- **head**: first node
- **tail**: last node (null node)
- **pointer**: reference to another node
- **node**: object containing data and pointer(s)
- **singly linked** lists only hold a pointer to the next node.
- **doubly linked** lists hold a pointer to the next and previous node.

	pros	cons
singly linked	uses less memory, simpler implementation	cannot easily access previous elements
double linked	can be traversed backwards	takes 2x memory

- linked lists vs array lists:
 - arrays are great at randomly accessing elements because they're indexed, but inserting/deleting elements gets troublesome (esp in larger arrays), since everything needs to be shifted if you insert/delete an element at any given index.
 - linked lists have an advantage when it comes to inserting/deleting, since you can just insert/delete a node at any give position and then adjust the **pointer/address** (no shifting required). but, because a linked list is a chain like structure, searching for an element is hard. you need to start at the head and work through the entire list until you find the element, whereas with arrays you just search with indices (the order in linked lists is determined by pointers and not by indices).
- operations:

LIST-SEARCH(L, k)

```

1   $x = L.head$ 
2  while  $x \neq \text{NIL}$  and  $x.key \neq k$ 
3       $x = x.next$ 
4  return  $x$ 
```

LIST-PREPEND(L, x)

```
1   $x.next = L.head$ 
2   $x.prev = NIL$ 
3  if  $L.head \neq NIL$ 
4       $L.head.prev = x$ 
5   $L.head = x$ 
```

- prepend is adding an element at the beginning of a list

LIST-INSERT(x, y)

```
1   $x.next = y.next$ 
2   $x.prev = y$ 
3  if  $y.next \neq NIL$ 
4       $y.next.prev = x$ 
5   $y.next = x$ 
```

- insert element anywhere in the list

LIST-DELETE(L, x)

```
1  if  $x.prev \neq NIL$ 
2       $x.prev.next = x.next$ 
3  else  $L.head = x.next$ 
4  if  $x.next \neq NIL$ 
5       $x.next.prev = x.prev$ 
```

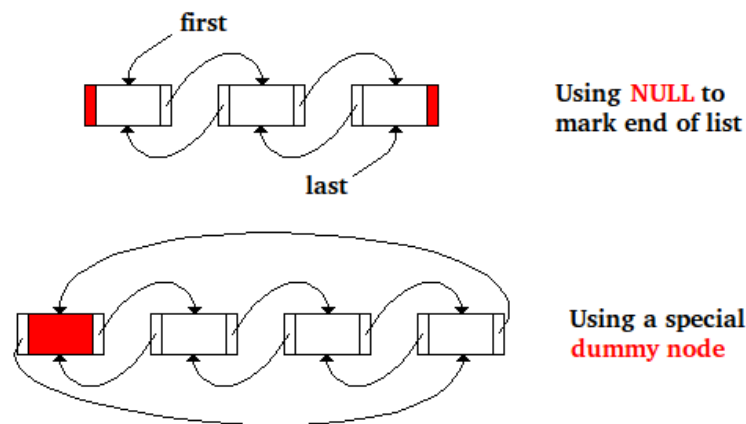
- time complexity:

	singly linked	doubly linked
search	$O(n)$	$O(n)$
insert at head	$O(1)$	$O(1)$
insert at tail	$O(1)$	$O(1)$

insert at middle	$O(n)$	$O(n)$
remove at head	$O(1)$	$O(1)$
remove at tail	$O(n)$	$O(1)$
remove in middle	$O(n)$	$O(n)$

▼ sentinel and circular lists

a **sentinel** is a dummy object added to lists to make them circular. it doesn't hold any value, but it makes operations on linked lists easier.



LIST-SEARCH'(L, k)

```

1   $L.nil.key = k$            // store the key in the sentinel to guarantee it is in list
2   $x = L.nil.next$          // start at the head of the list
3  while  $x.key \neq k$ 
4       $x = x.next$ 
5  if  $x == L.nil$            // found  $k$  in the sentinel
6      return NIL           //  $k$  was not really in the list
7  else return  $x$            // found  $k$  in element  $x$ 
```

- deleting and inserting with a sentinel:

LIST-DELETE'(x)

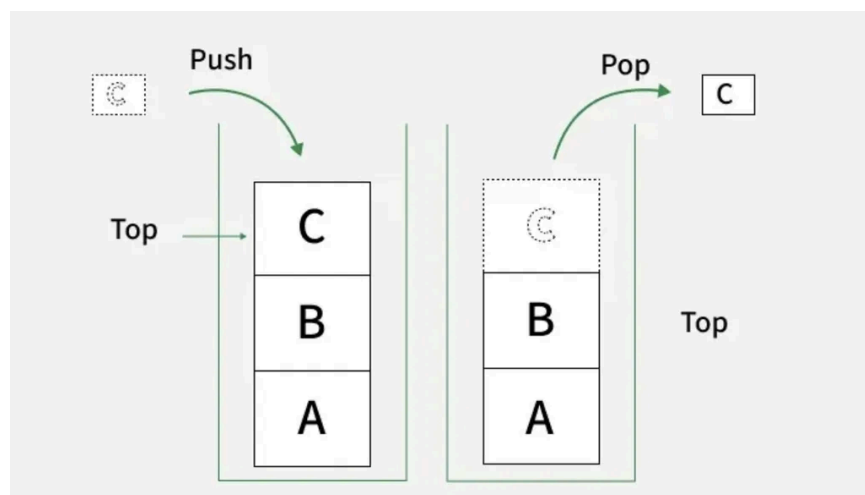
```

1   $x.prev.next = x.next$ 
2   $x.next.prev = x.prev$ 
```

LIST-INSERT'(x, y)

```
1  x.next = y.next
2  x.prev = y
3  y.next.prev = x
4  y.next = x
```

stacks



- a stack is a dynamic set that follows the **LIFO** (last in, first out) order. so the last inserted element is the first one to be removed, like stacking plates.

▼ stacks as arrays

- a stack of n elements $S[1:n]$ has a **top** and a **size**. S.top refers to the most recently inserted element (hence it's the "top") and S.size refers to the size n of the array. if the top exceeds the size, the stack overflows. if the stack is empty, so $S.top = 0$, then it underflows (unable to pop an empty stack).
 - **pop** = the deletion operation

POP(S)

```
1  if STACK-EMPTY( $S$ )
2      error "underflow"
3  else  $S.top = S.top - 1$ 
4      return  $S[S.top + 1]$ 
```

STACK-EMPTY(S)

```
1  if  $S.top == 0$ 
2      return TRUE
3  else return FALSE
```

- **push** = the insertion operation

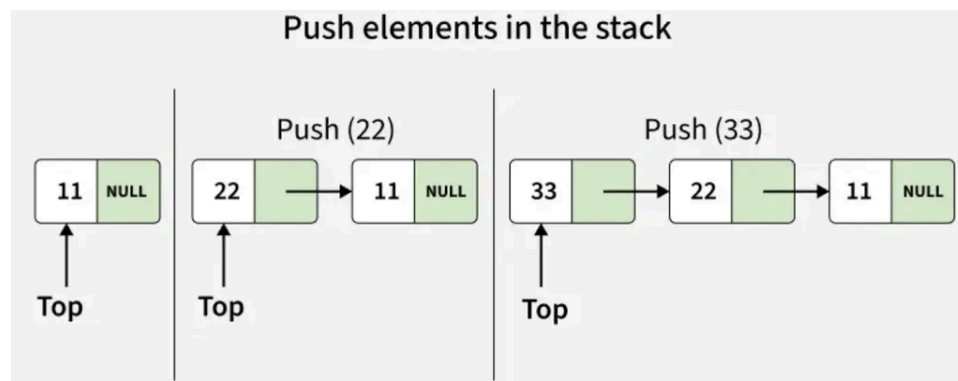
PUSH(S, x)

```
1  if  $S.top == S.size$ 
2      error "overflow"
3  else  $S.top = S.top + 1$ 
4       $S[S.top] = x$ 
```

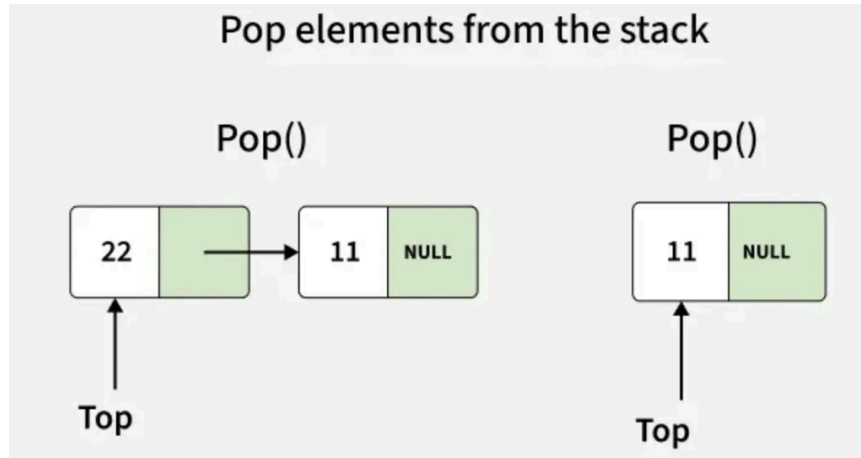
- time complexity: both operations are constant time functions, both inserting and removing an element takes a single memory allocation operation $\Rightarrow O(1)$

▼ stacks as linked lists

- similarly as arrays, a stack as a linked lists operates with pop and push.

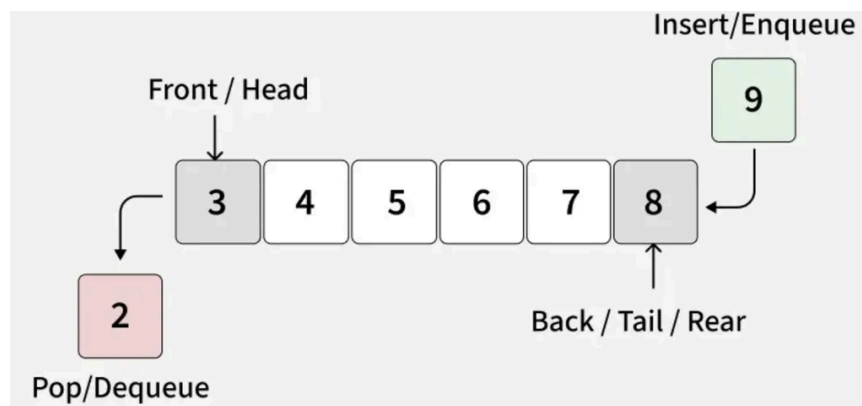


- when pushing a new node, insert it before the head(top) and not at the tail. so the head points at the newly inserted node, and the newly inserted node points at the node that is currently the head.



- popping works the same, you remove the current head(top) and then advance the head pointer to the next node in the stack.

queues



- a queue is a dynamic set that follows the FIFO (first-in, first-out) order, so elements are removed from the front first, just like an irl queue. similarly to linked lists, queues have a head and tail (front and back).

▼ queues as arrays

- a queue of n elements $Q(1:n)$ has a head, tail and a size of n elements (same definitions as stacks). if the head exceeds the tail (or tail exceeds the size), the queue overflows. if the queue is empty, it underflows.
 - **enqueue** = the insert operation
 - How to check for overflow? If $Q.head = Q.tail + 1$ or $Q.head = 1$ & $Q.tail = Q.size$

```

ENQUEUE( $Q, x$ )
1   $Q[Q.tail] = x$ 
2  if  $Q.tail == Q.size$ 
3       $Q.tail = 1$ 
4  else  $Q.tail = Q.tail + 1$ 

```

- **dequeue** = the delete operation
 - How to check for underflow? If $Q.head = Q.tail$

```

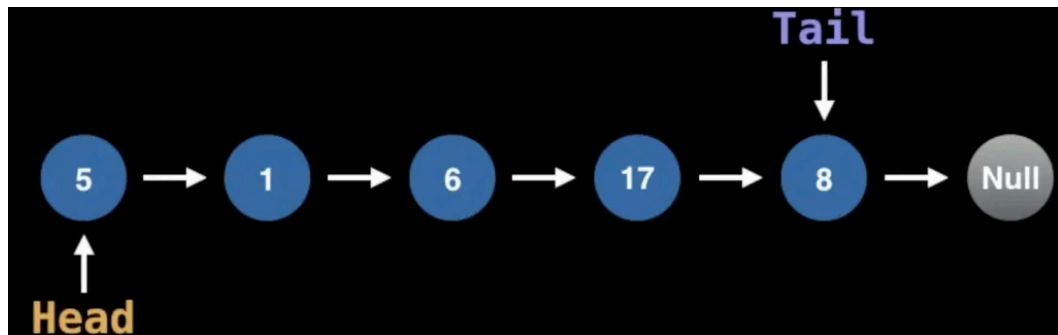
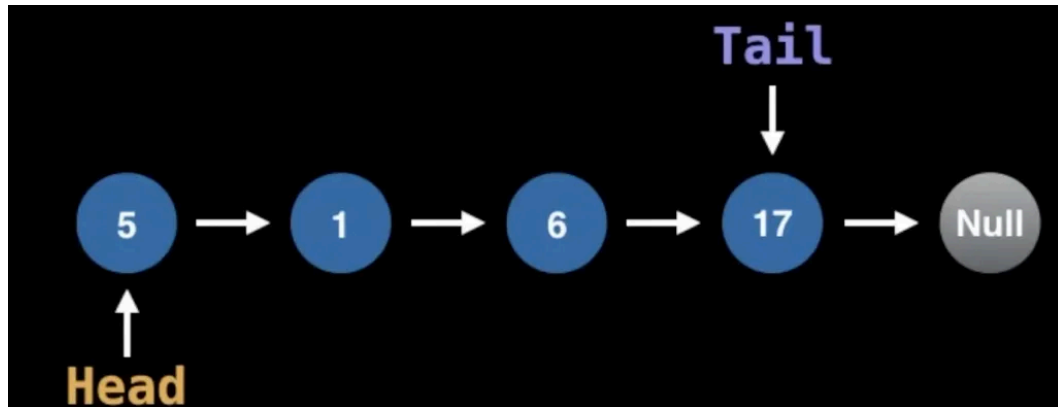
DEQUEUE( $Q$ )
1   $x = Q[Q.head]$ 
2  if  $Q.head == Q.size$ 
3       $Q.head = 1$ 
4  else  $Q.head = Q.head + 1$ 
5  return  $x$ 

```

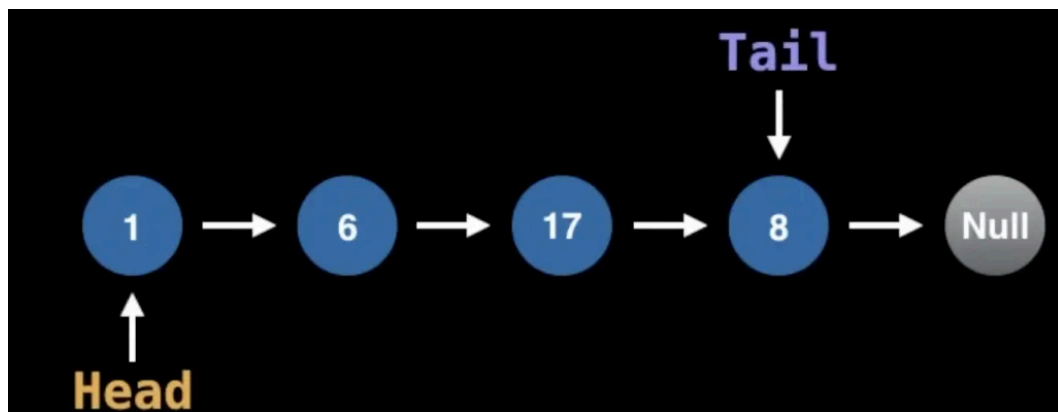
- time complexity: $O(1)$ for both enqueueing and dequeueing

▼ queues as linked lists

- when enqueueing a new node in a linked list, you push from the tail, so the newly inserted node is inserted at the "back" of the list and thus becomes the new node that points at the null node.



- when dequeuing, you remove from the head, so the current head is removed, and then the head pointer moves to the next node (making that node the new head).

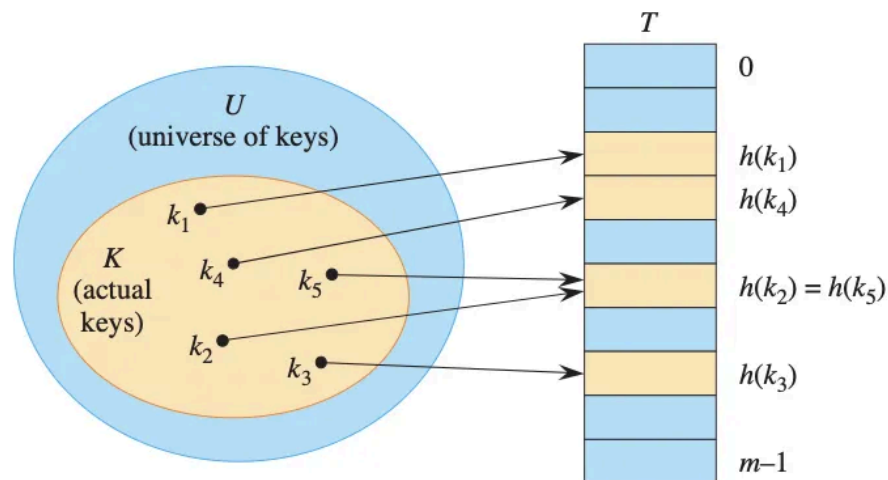


data structures

chapters 11-12

hash tables

- a hash table is a very efficient data structure that allow for fast searching/inserting/deleting of elements. It's a dictionary structure with key-value pairs.



- a **hash table** is a collection of key-value pairs where each key k is translated by a **hash function** h into the index of the value.
- the ratio between number of items stored and the size of the hash table is represented as

the load factor $a = n \text{ elements stored} / m \text{ slots}$.

▼ hash functions

a hash function is a function to map keys into a indices. the value that comes out of the hash function is also called **the hash value $h(k)$** .

- the ideal for any hash function is to satisfy **independent uniform hashing** where every key is equally likely to hash into any of the m available slots. a hash function should:
 - minimize collisions
 - give a uniform distribution of hash values (data spread uniformly across the hash table)
 - be easy to calculate
 - include a suitable method to resolve any collisions if they do occur.

- **static hashing** uses a single fixed function to compute indices. common approaches:
 - **the division method** is dividing the key k by the number of available slots m and then taking the remainder:
 - $h(k) = k \bmod m$
 - for alphanumeric keys, divide the sum of the ASCII codes in the key by n and then take the remainder.
 - **the multiplication method** has two steps, first multiply the key k by a constant A ($0 < A < 1$) and extract the fractional part of kA ($kA \bmod 1$). then, multiply this value by the number of slots m and take the floor of the result (round the final result down to the nearest integer):
 - $h(k) = \lfloor m \cdot (kA \bmod 1) \rfloor$
 - **the multiply shift method** is a practical version of the multiplication method. gemini explains cuz wth:

1. The Logic

Instead of using a fraction A (like 0.618), we use a large integer s .

- w : The number of bits in a computer word (e.g., 32 or 64).
- s : A large integer constant (usually an odd number).
- M : The table size, which **must** be a power of 2 (e.g., $M = 2^r$).

The formula is:

$$h(k) = (k \cdot s \bmod 2^w) \gg (w - r)$$

What is actually happening?

1. **Multiply**: You multiply the key k by s . The result can be up to $2w$ bits long.
2. **Modulo 2^w** : By storing the result in a standard w -bit register (like a `uint32`), the computer automatically throws away the high bits that overflow. This is your "fractional part."
3. **Shift Right ($\gg$)**: You shift the remaining bits to the right. This extracts the **most significant bits** of the product, which are the most "randomized" part of the multiplication.

2. A Practical Example (32-bit)

Let's say we have a 32-bit system ($w = 32$) and a hash table with **1024 slots** ($M = 2^{10}$, so $r = 10$).

1. **Pick s :** We use a constant that represents the Golden Ratio in 32-bit form: 2654435769 (this is $\approx \frac{\sqrt{5}-1}{2} \times 2^{32}$).
2. **Multiply:** Let $k = 12345$.
 - $12345 \times 2654435769 = 32,769,010,000,000\dots$ (Wait, this is too big!)
3. **Overflow:** The 32-bit register only keeps the bottom 32 bits.
 - The 32-bit result is: `0xB0F2A100` (in hex).
4. **Shift:** Since $w = 32$ and $r = 10$, we shift right by $32 - 10 = 22$ bits.
 - `0xB0F2A100 >> 22 = 707`

Result: Key 12345 is mapped to index 707.

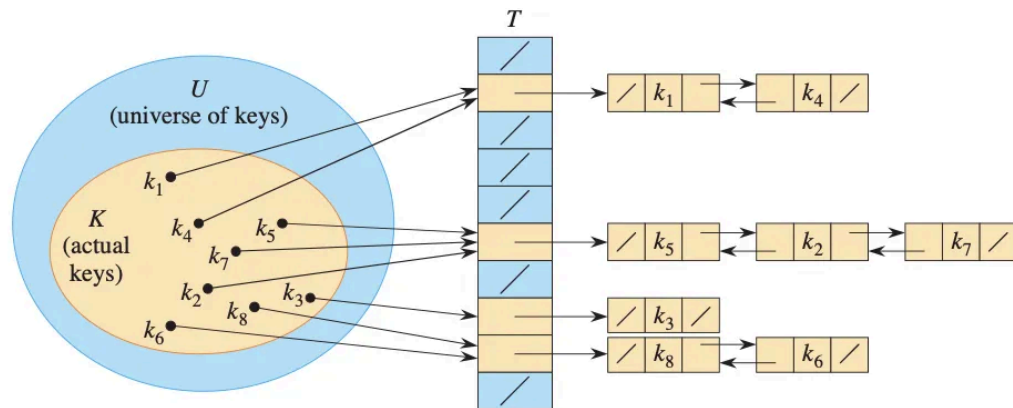
- to prevent choosing a set of keys that all hash to the same slot (and forcing a search to take $O(n)$ time which is worst-case behaviour), hash functions can be chosen randomly.
 - **universal hashing** involves picking a hash function at random from a bunch of carefully designed functions, ensuring the probability of collisions is kept to a minimum, which provides $O(1)$ average case performance for dictionary operations (insert, delete, search).

▼ collisions

- when two hash values are the same (and hash to the same slot in the table), it's called a **collision**. ultimately, this can be avoided by choosing the right hash function. but there are other ways to work around collisions:
 - **open addressing** is a way to avoid collisions by placing an item at an index that's not the calculated hash value. so if the hash value for a certain element A results in index 4, but that's already taken by another element B, you can place element A in a different slot (for example the next slot, or any slot that's empty). different methods for open addressing:
 - **linear probing** is using linear search to find a slot to insert an item, so the algorithm simply looks for the next available slot and places

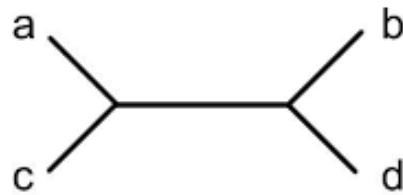
the collided key there. in best case, this takes $O(1)$ time, in worst-case it's $O(n)$

- **double hashing** makes use of a second hashing function to determine where to put the collided key.
- another (more efficient) way to work around collisions is by using linked lists (**chaining**), each slot in the hash table contains a **pointer** to a linked list and all elements with the same hash values will go in the linked list of that slot. so with the previous example, element A will hash to the linked list at index 4, and in the linked list it will be placed right after element B (B will point at A).
- insertion = $O(1)$
- deletion = $O(1)$
- search = $O(1 + \text{load factor } a)$

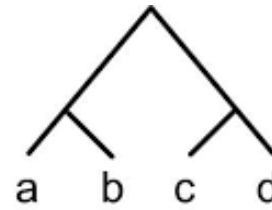


rooted trees

- rooted trees represent non linear relationships between elements. a rooted tree has a "**root**" node, a single node from which all other nodes descend. a=unrooted tree, b=rooted tree.

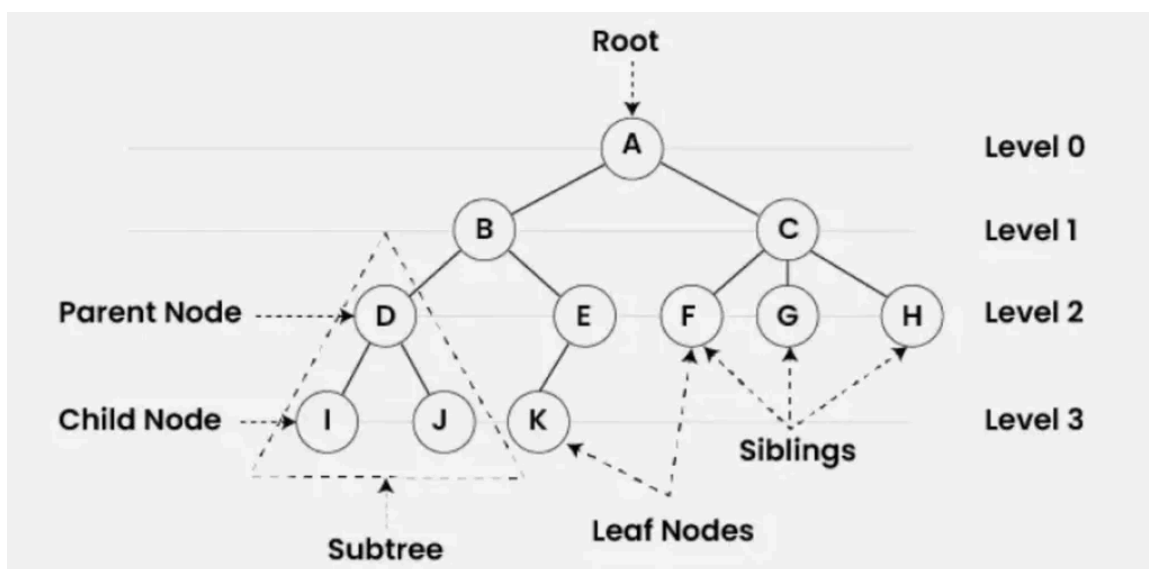


(a)



(b)

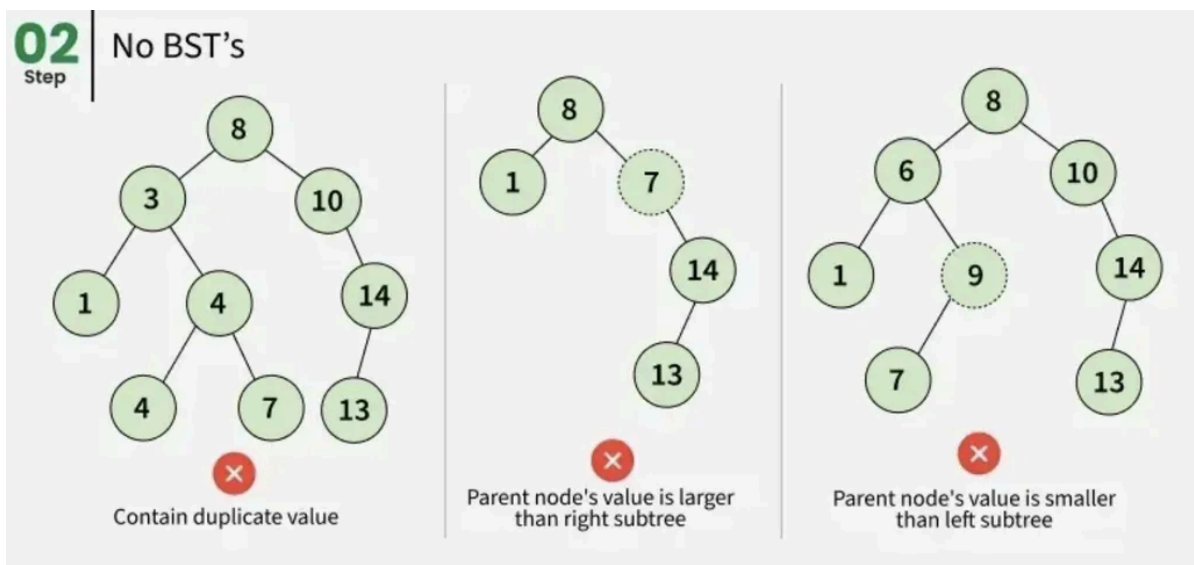
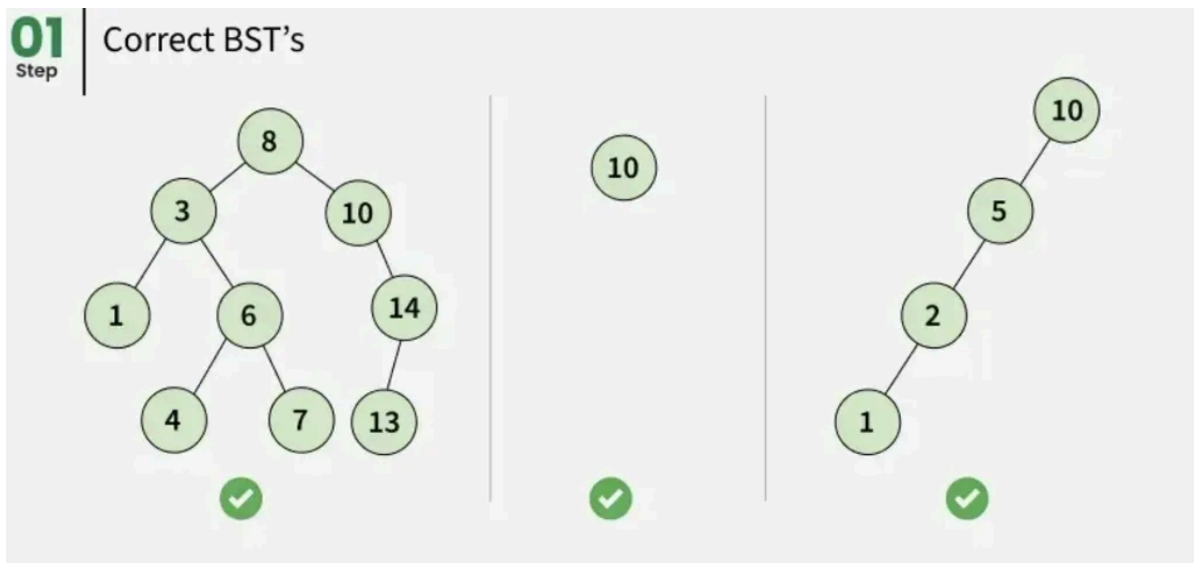
- nodes have a **parent-child** relationship:
 - if a node doesn't have children nodes, it's a **leaf** node.
 - nodes can be **siblings** to each other if they have the same parent node.
 - type of trees:
 - binary tree = every node has at most 2 children
 - ternary tree = every node has at most 3 children
 - n-ary tree = every node has at most n children
- trees have a **depth** and a **height**, which is determined by the number of **edges**(connection between nodes):
 - depth = number of edges below the parent node.
 - height = number of edges above the child node.
 - a smaller tree within a tree is a **subtree (can be a single node)**.



BST (binary search trees)

BST is a type of binary tree with a specific order:

- the left children of the parent nodes(the left subtree) are **smaller** than the parent node,
- the right children of the parent nodes(the right subtree) are **greater** than the parent node.



- time complexity for basic operations:

- average case: $O(\log n)$
- in the worst case: $O(n)$
- searching for an element in a BST, with x being a pointer to a node and k the key:

```

TREE-SEARCH( $x, k$ )
1  if  $x == \text{NIL}$  or  $k == x.\text{key}$ 
2      return  $x$ 
3  if  $k < x.\text{key}$ 
4      return TREE-SEARCH( $x.\text{left}, k$ )
5  else return TREE-SEARCH( $x.\text{right}, k$ )

ITERATIVE-TREE-SEARCH( $x, k$ )
1  while  $x \neq \text{NIL}$  and  $k \neq x.\text{key}$ 
2      if  $k < x.\text{key}$ 
3           $x = x.\text{left}$ 
4      else  $x = x.\text{right}$ 
5  return  $x$ 

```

- the **minimum** value of a BST is always the left-most node, and the **maximum** value of a BST is always the right most node.
 - finding the minimum/maximum takes $O(h)$ time where h is the height of the tree.

```

TREE-MINIMUM( $x$ )
1  while  $x.\text{left} \neq \text{NIL}$ 
2       $x = x.\text{left}$ 
3  return  $x$ 

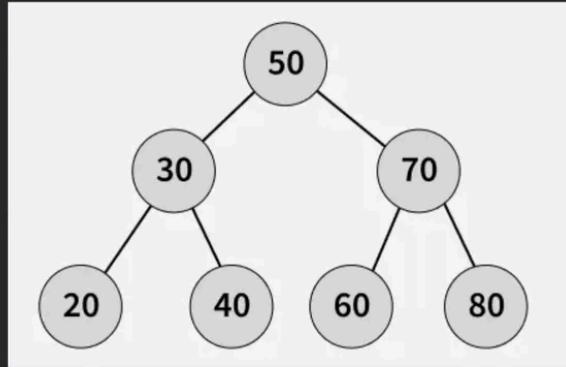
TREE-MAXIMUM( $x$ )
1  while  $x.\text{right} \neq \text{NIL}$ 
2       $x = x.\text{right}$ 
3  return  $x$ 

```

- given the **root** and an integer **key**:
 - the **predecessor** of a node is the node with the **largest** value *smaller than the key*.

- the **successor** of a node is the node with the **smallest** value *greater than* the key.

Input: key = 65



Output: [60, 70]

Explanation: In given BST the inorder predecessor of 65 is 60 and inorder successor of 65 is 70.

TREE-SUCCESSOR(x)

```

1  if  $x.right \neq \text{NIL}$ 
2      return TREE-MINIMUM( $x.right$ ) // leftmost node in right subtree
3  else // find the lowest ancestor of  $x$  whose left child is an ancestor of  $x$ 
4       $y = x.p$ 
5      while  $y \neq \text{NIL}$  and  $x == y.right$ 
6           $x = y$ 
7           $y = y.p$ 
8      return  $y$ 
  
```

- finding the successor/predecessor takes $O(h)$ time where h is the height of the tree.
- printing all keys in sorted order:
 - in order: print key of the root of a subtree between printing the values in its left subtree and printing those in its right subtree

```

INORDER-TREE-WALK( $x$ )
1  if  $x \neq \text{NIL}$ 
2      INORDER-TREE-WALK( $x.\text{left}$ )
3      print  $x.\text{key}$ 
4      INORDER-TREE-WALK( $x.\text{right}$ )

```

- pre order: print root before the values in either subtree
- post order: print root after the values in its subtrees

bst insert/delete

```

TREE-INSERT( $T, z$ )
1   $x = T.\text{root}$            // node being compared with  $z$ 
2   $y = \text{NIL}$              //  $y$  will be parent of  $z$ 
3  while  $x \neq \text{NIL}$       // descend until reaching a leaf
4       $y = x$ 
5      if  $z.\text{key} < x.\text{key}$ 
6           $x = x.\text{left}$ 
7      else  $x = x.\text{right}$ 
8   $z.p = y$                // found the location—insert  $z$  with parent  $y$ 
9  if  $y == \text{NIL}$ 
10      $T.\text{root} = z$        // tree  $T$  was empty
11 elseif  $z.\text{key} < y.\text{key}$ 
12      $y.\text{left} = z$ 
13 else  $y.\text{right} = z$ 

```

TRANSPLANT(T, u, v)

```
1  if  $u.p == \text{NIL}$ 
2       $T.root = v$ 
3  elseif  $u == u.p.left$ 
4       $u.p.left = v$ 
5  else  $u.p.right = v$ 
6  if  $v \neq \text{NIL}$ 
7       $v.p = u.p$ 
```

TREE-DELETE(T, z)

```
1  if  $z.left == \text{NIL}$ 
2      TRANSPLANT( $T, z, z.right$ )           // replace  $z$  by its right child
3  elseif  $z.right == \text{NIL}$ 
4      TRANSPLANT( $T, z, z.left$ )             // replace  $z$  by its left child
5  else  $y = \text{TREE-MINIMUM}(z.right)$         //  $y$  is  $z$ 's successor
6      if  $y \neq z.right$                      // is  $y$  farther down the tree?
7          TRANSPLANT( $T, y, y.right$ )        // replace  $y$  by its right child
8           $y.right = z.right$                  //  $z$ 's right child becomes
9           $y.right.p = y$                      //  $y$ 's right child
10     TRANSPLANT( $T, z, y$ )                   // replace  $z$  by its successor  $y$ 
11      $y.left = z.left$                        // and give  $z$ 's left child to  $y$ ,
12      $y.left.p = y$                          // which had no left child
```

(sorting) algorithms

chapters 2, 6-8

loop invariants

a loop invariant is a condition that is true before, during and after each iteration of a loop. it satisfies three conditions:

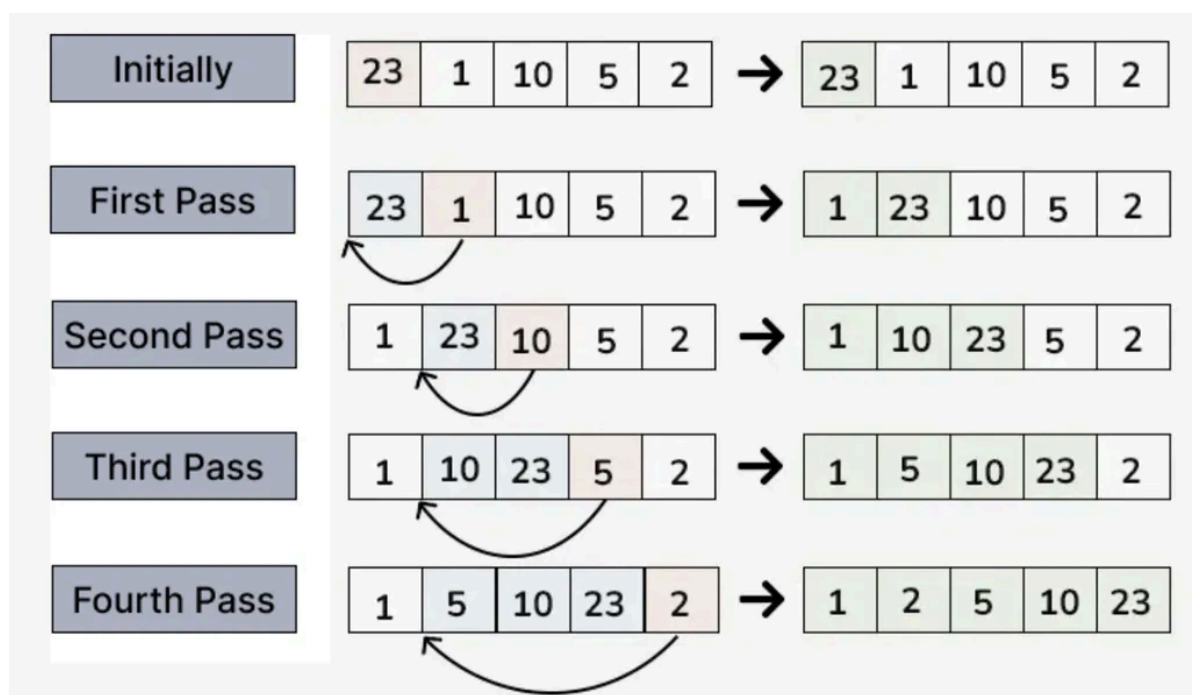
- **initialization**: true before the first iteration of the loop
- **maintenance**: if true before an iteration of the loop, then also true after the iteration
- **termination**: after the loop terminates, the invariant should tell us something useful about the algorithm.

loop invariants capture the key facts of why code works and help us understand why an algorithm is correct.

insertion sort

insertion sort is a simple algorithm for sorting elements by iteratively inserting each element of the unsorted part of the array in the sorted part of the array. it's best used for shorter arrays.

- best case: $O(n)$ if the list is already sorted
- average case and worst case: $O(n^2)$ if the list is randomly sorted and if the list is reverse ordered



▼ pseudocode

- the pseudocode uses 1-based indexing, so the array starts at index 1.

```
INSERTION-SORT( $A, n$ )
1  for  $i = 2$  to  $n$ 
2       $key = A[i]$ 
3      // Insert  $A[i]$  into the sorted subarray  $A[1 : i - 1]$ .
4       $j = i - 1$ 
5      while  $j > 0$  and  $A[j] > key$ 
6           $A[j + 1] = A[j]$ 
7           $j = j - 1$ 
8       $A[j + 1] = key$ 
```

- outer-loop (line 1):
 - start at the second element of the array and iterate through each one until you reach the end of the array n .
 - set the element at the current index i to be the key , so it doesn't get overwritten later on in the algorithm (line 2)
 - then set a pointer j at the index **left** of the current element: $j = i - 1 = 2 - 1 = 1$ (line 4), so j for the first iteration is 1 (the first element)
 - the inner loop (line 5):
 - now check if $j > 0$ ($j = -1$ doesn't exist because the very first index is 1), and then check if the element at index j : $A[j]$ is greater than the key. if both conditions are satisfied:
 - move the element at index j one position to the right (line 6)
 - now move the pointer j one position to the left to check the elements before it (line 7). repeat the above steps for as long as the while loop holds.
 - if, by decrementing j , the while loop breaks, go to line 8.

- **note:** when the while loop breaks, j has a *new* value, because in line 7 j is moved one position to the left. in this first iteration, j now becomes $j = 1-1 = 0$, breaking the while loop.
- last step is to place the key that was stored earlier at one index to the **right** of j , since this is now an empty slot. in this iteration, that index would be $j+1 = 0+1 = 1$, so the key is placed at the index 1 (line 8).
- now increment i and continue until the end of the array (:start again from line 2).

▼ loop invariant

<https://youtu.be/SEIE4RIAj0?si=X7vYCz1UGhQh-nMf>

the subarray $A[1:i-1]$ is sorted at the start of each iteration of the outer loop (for loop in line 1-8).

initialization: before the first iteration, $i = 2$. so, the subarray $A[1:i-1]$ consists of the elements at $A[1:1]$, which is just a single element $A[1]$. an array with just a single element is already in sorted order, therefore the loop invariant holds.

maintenance: during the start of an iteration, the subarray $A[1:i-1]$ is in sorted order. during the while loop, the algorithm shifts the elements to the left of the key $A[i]$ (the elements at $A[i-1]$, $A[i-2]$, etc) one position to the right until it finds the correct position for $A[i]$, and then inserts it at that position. once the key is inserted, the subarray $A[1:i]$ now contains the original elements, but in sorted order. before the next iteration, $A[1:i-1]$ is now sorted, so the loop invariant holds.

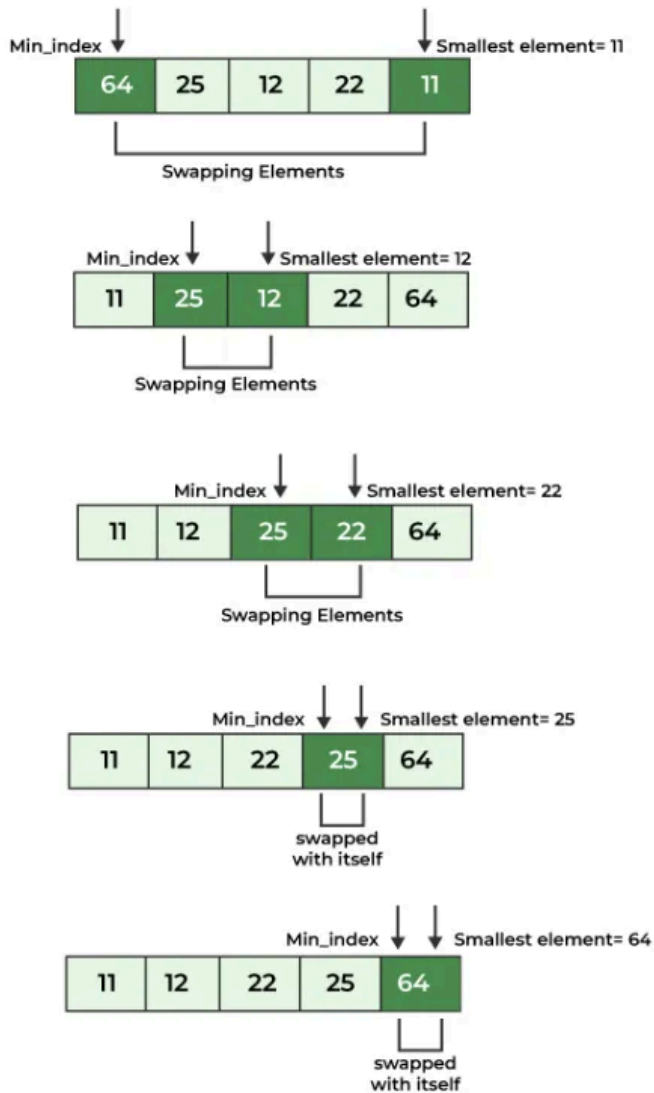
termination: the loop terminates when i exceeds the size of the array n , that is $i=n+1$. so, the subarray $A[1:i-1]$ then becomes $A[1:n+1-1]$ which is just the entire array $A[1:n]$, and this entire subarray consists of the original elements in $A[1:n]$ but in sorted order, therefore the loop invariant holds.

selection sort

selection sort is a comparison based algorithm that sorts an array by repeatedly taking the smallest element from the unsorted subarray, and swapping it with the

first unsorted element of the array.

- time complexity: $O(n^2)$ since there are two for loops



▼ pseudocode

1-based indexing:

```
SelectionSort(A, n) // n= size of the array
for i = 1 to n-1
    min_element = i

    for j = i+1 to n
```

```
        if A[j] < A[min_element]
            min_element = j

    swap(A[i], A[min_element])
```

- outer loop:
 - start at the first element of the array and iterate through each one until $n-1$, because the last element $A[n]$ is an unsorted subarray of just a single element, which is always sorted.
 - set the current index i as variable `min_element`, which at the first iteration is $i=1$ (first element).
 - inner loop:
 - then loop through the unsorted subarray to compare the element at `min_element` to every other element in this subarray. so start at $j=i+1=2$ (the second element) and iterate through each one until the end of the array n .
 - then compare the element at the current index j with the element that's at the `min_element`. if $A[j]$ is smaller than $A[\text{min_element}]$, then update `min_element` to j . if however the if-statement is not satisfied, so $A[j]$ is not smaller than $A[\text{min_element}]$, `min_element` is not updated. so in this iteration `min_element` will become 2 if the if-statement is met.
 - now move on to the next element by incrementing j and with the new value of `min_element`, so now $j=3$ and `min_element=2`.
 - check the if-statement again, if $A[j]$ is smaller than $A[\text{min_element}]$, update `min_element` again to j , and then iterate again and again until the end of the array is reached.
 - after the inner loop breaks, swap the element at the current index i (in this iteration that's 1) with the element at `min_element`.
 - then go back to the outer loop and go through all the steps again from the next i .

▼ loop invariant

at the start of each iteration of the outer for loop, the subarray $A[1:i-1]$ is sorted and has the $(i-1)$ smallest elements of the entire array.

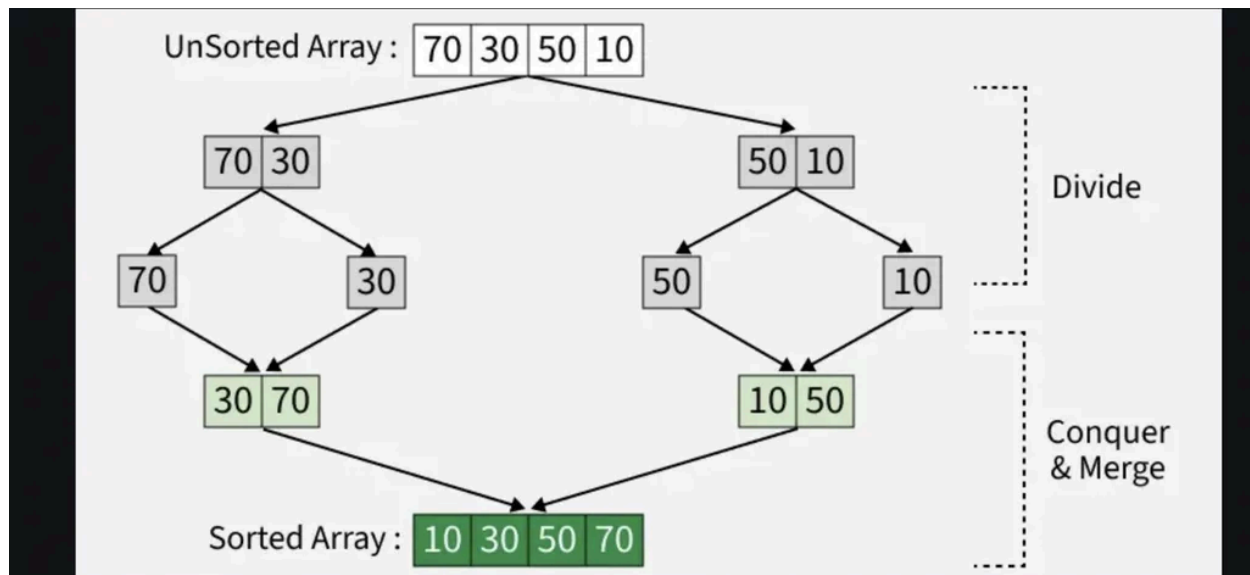
initialization: before the first iteration: $i=1$. so the subarray consists of $A[1:0]$, which is an empty subarray, and thus sorted and contains the smallest elements of the entire array (since it's empty). so the loop invariant holds.

maintenance: during the start of an iteration, $A[1:i-1]$ is sorted and has the smallest $i-1$ elements of the array. during the inner loop, the algorithm searches the remaining part of the array $A[i:n]$ to find the smallest element. when the smallest element is found it's swapped with $A[i]$. so now $A[1:i]$ contains the smallest elements of the original array in sorted order. when i increments for the next iteration, the loop invariant still holds.

termination: the outer loop runs until $i=n$, so when the loop terminates, $A[1:n-1]$ contains the $n-1$ smallest elements. and because $n-1$ is sorted and contains the smallest elements of the original array, the very last element $A[n]$ must be the largest element of the array, therefore the entire array $A[1:n]$ is sorted.

merge sort

merge sort follows the divide and conquer approach to sort arrays. this algorithm works recursively, it divides the entire array into smaller halves until dividing is not possible anymore (so it divides the entire array into halves consisting of 1 element), and then merges everything back together until the entire array consists of all original elements in sorted order.



Here's a step-by-step explanation of how merge sort works:

1. **Divide:** Divide the list or array recursively into two halves until it can no more be divided.
2. **Conquer:** Each subarray is sorted individually using the merge sort algorithm.
3. **Merge:** The sorted subarrays are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.

- time complexity: $O(n \log n)$ for all cases.

<https://youtu.be/3j0SWDX4AtU?si=M-URix9yN34liYbz>

▼ pseudocode

merge sorts consists of two parts, the merge algorithm and the recursive sorting algorithm.

<https://youtu.be/6pV2IF0fgKY?si=noFfcnevgS3xFPtU>

- 0 based indexing so array starts at index 0 and not 1:

```

MERGE( $A, p, q, r$ )
1   $n_L = q - p + 1$            // length of  $A[p : q]$ 
2   $n_R = r - q$                // length of  $A[q + 1 : r]$ 
3  let  $L[0 : n_L - 1]$  and  $R[0 : n_R - 1]$  be new arrays
4  for  $i = 0$  to  $n_L - 1$  // copy  $A[p : q]$  into  $L[0 : n_L - 1]$ 
5       $L[i] = A[p + i]$ 
6  for  $j = 0$  to  $n_R - 1$  // copy  $A[q + 1 : r]$  into  $R[0 : n_R - 1]$ 
7       $R[j] = A[q + j + 1]$ 
8   $i = 0$                      //  $i$  indexes the smallest remaining element in  $L$ 
9   $j = 0$                      //  $j$  indexes the smallest remaining element in  $R$ 
10  $k = p$                      //  $k$  indexes the location in  $A$  to fill
11 // As long as each of the arrays  $L$  and  $R$  contains an unmerged element,
    // copy the smallest unmerged element back into  $A[p : r]$ .
12 while  $i < n_L$  and  $j < n_R$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
18      $k = k + 1$ 
19 // Having gone through one of  $L$  and  $R$  entirely, copy the
    // remainder of the other to the end of  $A[p : r]$ .
20 while  $i < n_L$ 
21      $A[k] = L[i]$ 
22      $i = i + 1$ 
23      $k = k + 1$ 
24 while  $j < n_R$ 
25      $A[k] = R[j]$ 
26      $j = j + 1$ 
27      $k = k + 1$ 

```

- first initialize the left and right arrays and copy the original elements into it (lines 1-7)
 - p = beginning of array, q = middle of array, r = end of array
 - $n_L = q - p + 1$ is the length of the left side of the original array, so middle - beginning and then +1 because this code uses 0-based indexing. (line 1)
 - $n_R = r - q$ is the length of the right side of the original array, so end - middle. (line 2)

- the two for-loops (line 4-6) copy each element from the original array into the corresponding L or R array.
- line 8-10, set the indices for iterating through L, R and A. i is the index in L, j is the index in R and k is the index in A (original array)
- then loop through each element in L and R to determine which one is the smallest, then copy that element to A, and then increment both i and j and continue until one of the arrays is exhausted (line 12-18)
 - while i and j are within the length of L and R respectively, check if L[i] is smaller than/equal to R[j]. (line 12-13)
 - if L[i] is smaller, then A[k] = L[i], and increment i by 1. so in the L array we move to the next index, in the R array we're still at the same index. (line 13-15)
 - line 16-17 if R[j] is smaller, then A[k] = R[j], and increment j by 1. in the R array we're at the next index and in the L array we're at the same index.
 - at the end of each iteration, move k by 1 to the next slot in A.
- when either one of L and R has been gone through entirely, copy all remaining elements of the other array to the last slot(s) of A. since these are the last remaining elements, they must be the largest elements of both L and R, so there's no need to compare them again. (line 20-27)

https://youtu.be/mB5HXBb_HY8?si=vZwwfQOi6hENhKSt

```

MERGE-SORT( $A, p, r$ )
1  if  $p \geq r$                                 // zero or one element?
2      return
3   $q = \lfloor (p + r) / 2 \rfloor$                     // midpoint of  $A[p : r]$ 
4  MERGE-SORT( $A, p, q$ )                          // recursively sort  $A[p : q]$ 
5  MERGE-SORT( $A, q + 1, r$ )                      // recursively sort  $A[q + 1 : r]$ 
6  // Merge  $A[p : q]$  and  $A[q + 1 : r]$  into  $A[p : r]$ .
7  MERGE( $A, p, q, r$ )
  
```

- merge-sort has 3 parameters: A=array, p=beginning of array, r=end of array

- if p is greater than/equal to r , which means there's either only 1 element in the array or it's an empty array, the algorithm stops. (line 1-2)
- initialize the midpoint of A , which is the entire length of the array $p+r$ divided by 2. (line 3)
- then the algorithm calls itself again on the left side of the array (line 4), and on the right side of the array (line 5). this happens recursively, so the left side is then divided again into a left and right side, and the same happens to the right side, until the algorithm has divided the entire original array into smaller arrays consisting of only 1 element.
 - the algorithm first divides each left side into single elements, and then merges these halves together in sorted order, THEN it divides each right side into single elements and merges those together. when there's two halves again (1 left side and 1 right side), these two are merged and this signals the last merge before the algorithm terminates.

▼ loop invariant

at the start of each iteration of the while loop (merge line 12-18) the subarray $A[p:k-1]$ is sorted and has the smallest elements of subarrays L and R .

initialization: $k=p$ (line 10), so $A[p:k-1] = A[p:p-1]$, which is an empty array. an empty array is sorted by default and consists of the smallest elements of L and R . so L and R are also empty and thus sorted.

maintenance: during each iteration, $L[i]$ and $R[j]$ are compared and the smallest one is copied to $A[k]$. now $A[p:k]$ consists of the smallest elements in sorted order. the next element at either $L[i]$ or $R[j]$ now has to be the smallest one.

termination: the loop terminates when either L or R runs out, which means at this point all smallest elements are in $A[p:k-1]$. the remaining elements in either L or R must be the largest elements that need to be copied into A . therefore after termination, $A[p:k-1]$ is completely sorted.

heaps and heap sort

heap sort is a comparison based sorting algorithm based on binary heaps (tree data structure). it's an optimized version of selection sort: it finds the maximum/minimum element and swaps it with the first element repeatedly.

<https://youtu.be/HqPJF2L5h9U?si=BL3LO8kxn-9sdFuy>

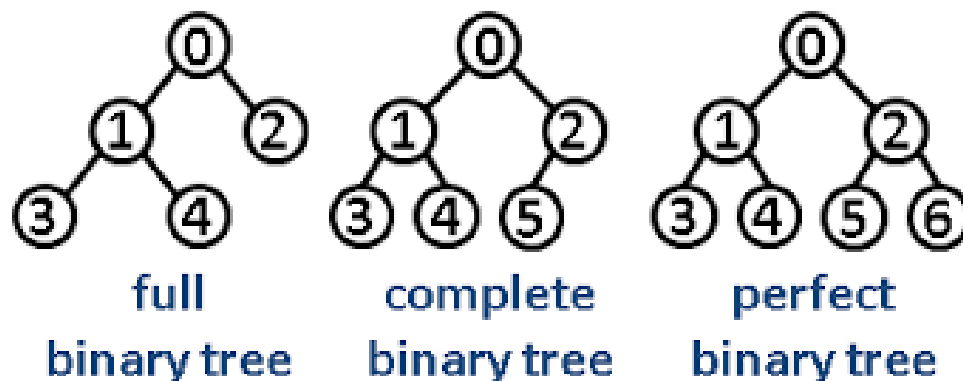
▼ full vs complete binary tree

there's a distinction between a *full* binary tree and a *complete* binary tree. a full binary tree has nodes with exactly 0 or 2 children, so no node can have only 1 child. a complete binary tree doesn't need to have the maximum number of nodes, but it's filled on all levels from left to right except for the lowest possible level (height of a complete binary tree is $\log n$). in other words, if the tree were put in array notation, there are no gaps between the slots.

the main differences:

- a full tree has exactly 0 or 2 children for every internal node, and every leaf node has 0 children. so there are NO nodes with only 1 child. leaf nodes can be on different depths of the tree.
- a complete tree is completely filled at every level, except possibly for the last level, which must be filled from left to right. a node with 1 child is possible, but this must be the very last node of the tree and it must be only a left child. easy way to check is by putting the tree in array notation, if there are no gaps in the array (no empty slots in the middle) the tree is complete.

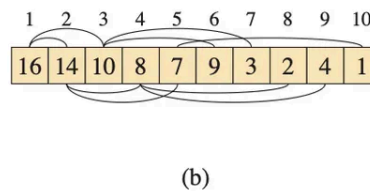
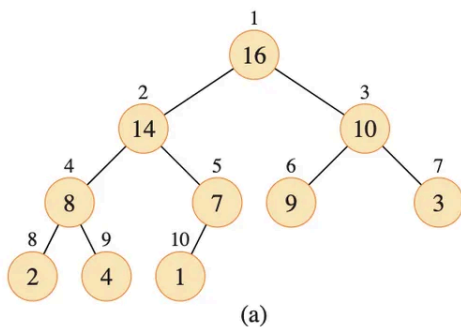
a tree can be both full and complete, then it's called a perfect binary tree (nr of nodes: $2^{(h+1)}-1$).



▼ binary heap

a binary heap has two conditions:

- it's a complete binary tree
- the nodes are ordered in a max or min heap structure:
 - **max heap**: the parent node's value is **greater than/equal to** all its descendants, so the root has the largest value of the entire heap. (pic below)
 - **min heap**: the parent node's value is **smaller than/equal to** all its descendants, so the root has the smallest value of the entire heap.



- if a node is at index i :
 - left child: $2i$
 - right child: $2i + 1$
 - parent: $i/2$ floor value (rounded down)

```
PARENT( $i$ )  
1  return  $\lfloor i/2 \rfloor$   
  
LEFT( $i$ )  
1  return  $2i$   
  
RIGHT( $i$ )  
1  return  $2i + 1$ 
```

heapify

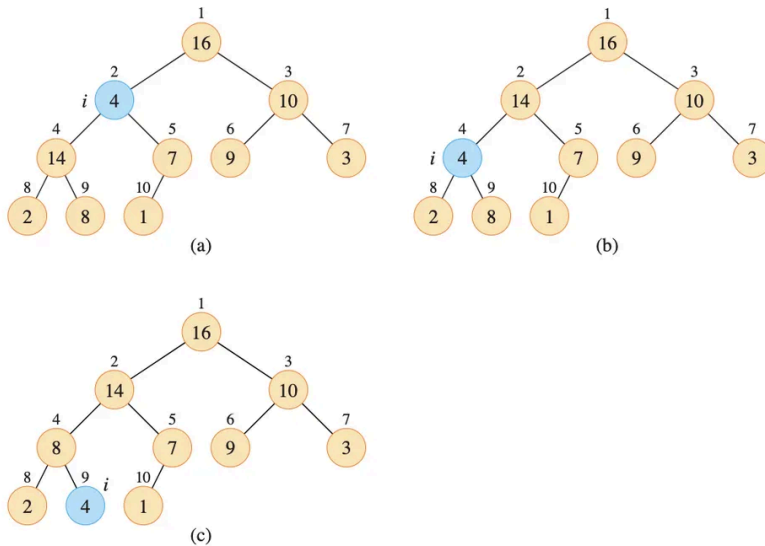
heapify is the process of turning a heap into a max or min heap

- time complexity: $O(\log n)$

```
MAX-HEAPIFY(A, i)
l = 2i //left child
r = 2i + 1 //right child
if l <= A.heap_size and A[l] > A[i] //compare left child with
current element
    largest = l //if left child is greater than A[i], left ch
ild index is largest
else largest = i //otherwise current element index is largest

if r <= A.heap_size and A[r] > A[largest] //now compare right
child with largest
    largest = r

if largest != i //if largest is not the current index:
    swap(A[i], A[largest]) //swap current element with elemen
t at 'largest' index
    MAX-HEAPIFY(A, largest) //the now swapped largest element
might break the heap
    //property again, so call the function again to perform m
ax-heapify.
```



- the min-heapify algorithm is the same, but instead of checking for the largest element you check for the smallest: operators flip the other way, so $A[l] < A[i]$ and $A[r] < A[\text{smallest}]$.

heap sort

heap sort has two main steps:

- for a given set of elements, create a heap $\rightarrow O(n)$ time
 - then delete all elements from the created heap $\rightarrow O(\log n)$ time
- total time = $O(n \log n)$

when deleting elements from a heap, as seen in the pseudocode, the deleted element is placed at the end of the array. by going through all elements like this, the array ends up sorted from smallest to largest element in a max heap (largest to smallest in a min heap). this is the main idea of heap sort. below pseudocode for max heap. heap sort for a min heap works the same, but instead of max-heapify we call min-heapify.

```
BUILD-MAX-HEAP(A, n)
```

```
A.heap_size = n //initialize the heap size
```

```
for i = [n/2] downto 1 //start from leaves and work up to the
```

root

```
MAX-HEAPIFY(A, i) //rearrange the heap as a max heap
```

- why start at $i = \lfloor n/2 \rfloor$:
 - leaves of a heap are located at index $\lfloor n/2 \rfloor + 1$. a leaf node doesn't have children and therefore is a valid min/max heap of size 1. so running heapify on a leaf node doesn't do anything since it doesn't have left or right children to compare. which is why the loop starts at $i = \lfloor n/2 \rfloor$ to save time.

```
HEAP-SORT(A, n)
```

```
BUILD-MAX-HEAP(A, n) //create a heap for elements n
```

```
for i = n downto 2 // start from the last heap node and work to i=2
```

```
    swap(A[1], A[i]) //swap last element of the heap with the root
```

```
    A.heap_size = A.heap_size - 1 //last element is deleted so shrink heap size
```

```
    MAX-HEAPIFY(A, 1) //restore max-heap property
```

- loop from $i = n$ down to $i = 2$ because the first element at $i = 1$ must be the smallest element if all other elements are sorted already, so including $i = 1$ is redundant.
- call max-heapify on index 1 (the root) since the root gets deleted and the new root is 'out of place'.

priority queue

a priority queue is a type of queue that stores elements with a certain priority

- elements with higher priority are removed first before the ones with lower priority
- new elements are added based on priority as well

in context of heaps:

- min-heaps allow quick access to the smallest element

- elements are dequeued from smallest to largest element
- max-heaps allow quick access to the largest element
 - elements are dequeued from largest to smallest element

inserting an element in a heap happens from the leaf, then this new element is compared to its parents and pushed up to the correct place. so leaf $\Rightarrow$ root. inserting in a heap takes:

- $O(\log n)$ maximum, depends on the height of the tree (if the inserted element needs to be pushed up the tree).

```
MAX-HEAP-INSERT(A, key)
A.heap_size = A.heap_size + 1 //add the new element by
increasing the heap size
i = A.heap_size //set index at the last element (newly
inserted one)
A[i] = key // set A[i] as the key
while i > 1 and A[i/2] < A[i] //check index is not at r
oot, and key element is

//greater than parent
    swap(A[i], A[i/2]) //if key is greater than parent,
swap them
    i = i/2 //compare key with the next parent
```

- if min heap instead of max heap, the comparison operator is flipped, but the steps are the same.

deleting an element in a heap starts from the root (if you have items stacked as a pyramid, you remove the item from the top so the pyramid still holds its shape), then rearrange the tree so it maintains the max/min heap property (heapify). so root $\Rightarrow$ leaf. deleting in a heap takes:

- $O(\log n)$ maximum, depends on the height of the tree.

```

EXTRACT-MAX(A)
if A.heap_size < 1
    return //empty tree
max = A[1] //the largest element is the root
A[1] = A[A.heap_size] //remove the root by placing it at the end of the array !!
MAX-HEAPIFY(A, 1) //rearrange the tree to maintain max-heap property
return max //return the just deleted element for processing (priority queue)

```

- when deleting elements, the deleted element is placed at the end of the array (important note for heap sort)

insertion/deletion in min and max heaps work with the same steps, only difference is the comparison to push an element to the root/leaf. in a max heap, the parent should have the greater value compared to the children. in a min heap, the parent should have the smaller value compared to the children.

to change the priority of an element in a priority queue, use the increase-key algorithm. increasing the key of an element takes:

- $O(\log n)$

```

MAX-INCREASE-KEY(A) //for a max priority queue
if key < A[i]
    return //if new key is smaller than current key -> error
A[i] = key //set current node as key
while i > 1 and A[i/2] < A[i] //compare current node with its parent
    swap(A[i], A[i/2]) //if child greater than parent, then swap
    i = i/2 //continue to next parent

```

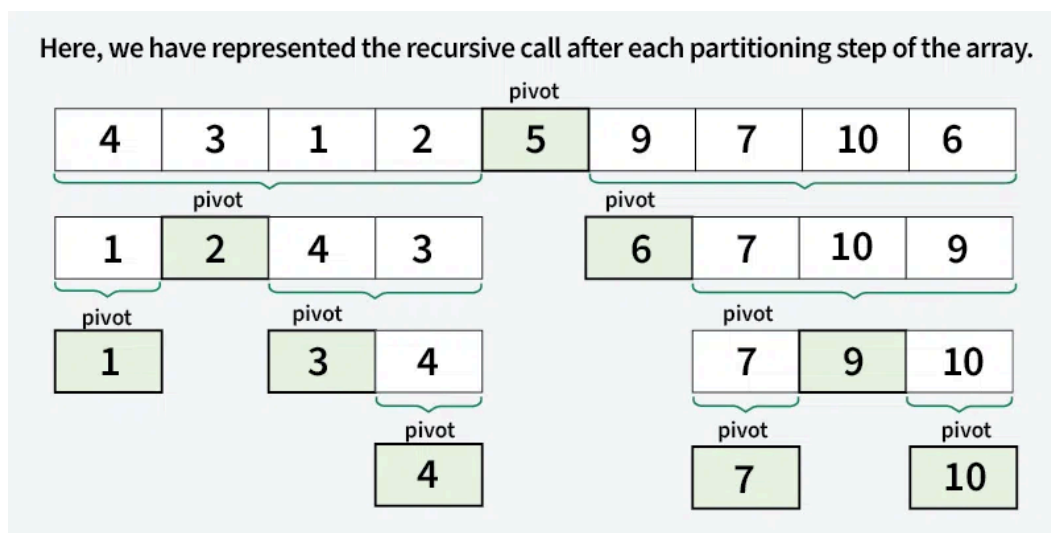
- for a min-priority queue, flip the comparison operators

quick sort

<https://youtu.be/Vtckgz38QHs?si=s-kZFEP9Gwn919pR>

quick sort is a recursive sorting algorithm based on divide and conquer method, it consists of 3 main steps:

- choose a pivot
- rearrange the array so that all elements smaller than the pivot is on its left and elements larger than the pivot are on its right
- recursively apply same steps to the created subarrays to sort the entire array
 - base case: recursion stops when only 1 element is left in the subarray, because a single element is already sorted.



time complexity:

- best case $O(n \log n)$
- average case $O(n \log n)$
- worst case $O(n^2)$ if already sorted

▼ pseudocode

QUICKSORT(A, p, r)

```
1  if  $p < r$ 
2      // Partition the subarray around the pivot, which ends up in  $A[q]$ .
3       $q = \text{PARTITION}(A, p, r)$ 
4      QUICKSORT( $A, p, q - 1$ ) // recursively sort the low side
5      QUICKSORT( $A, q + 1, r$ ) // recursively sort the high side
```

- p = beginning of array, r = end of array
- set the pivot q by calling the partition algorithm on the array
 - partition algorithm will sort the array by moving elements smaller than the pivot to its left and elements larger than the pivot to its right
- recursively sort the left side (low side) and the right side (high side) of the array (pivot excluded)

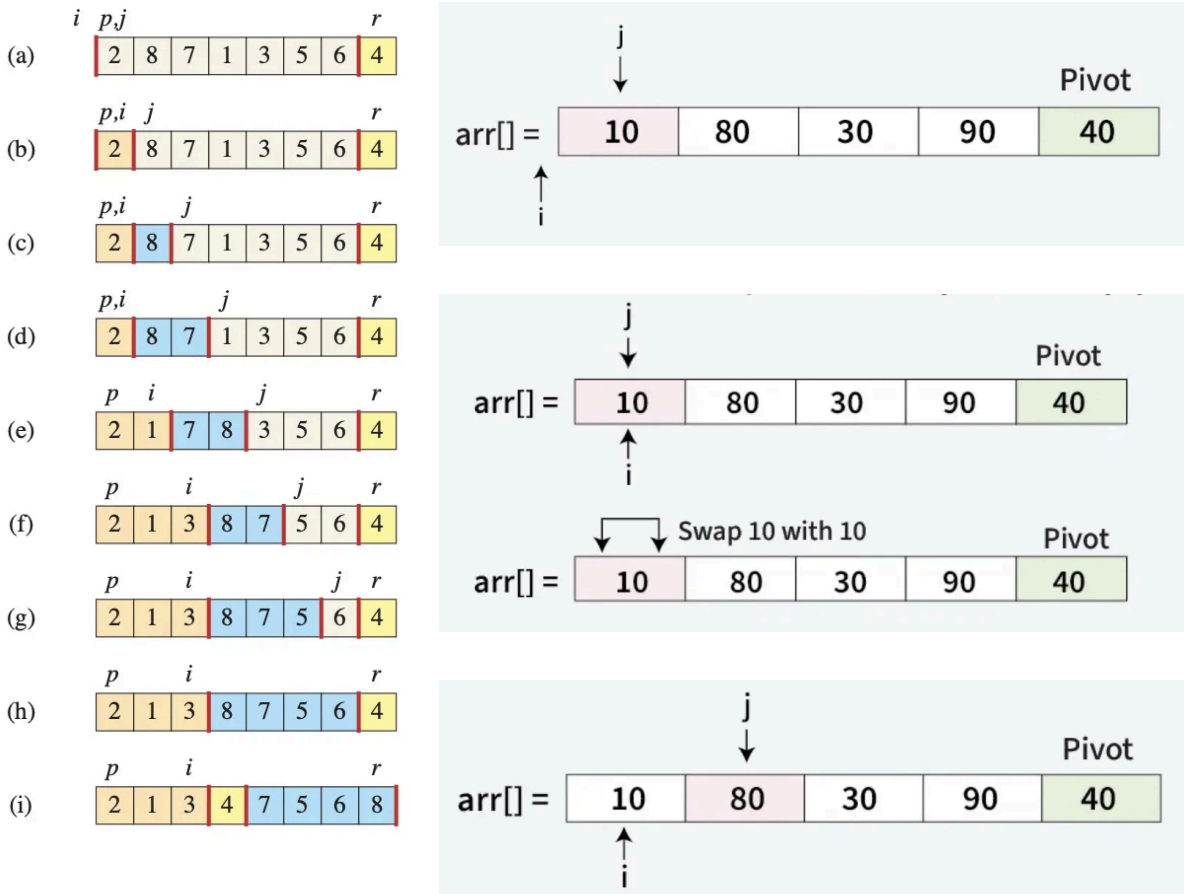
PARTITION(A, p, r)

```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```

- set the last element as the pivot x
- to compare elements to the pivot, we need two indices i and j
 - i is set at -1 , before the beginning of the array, j is set at the beginning of the array, so i and j are next to each other at the start
 - then iterate through the entire array except for the pivot
 - if $A[j]$ is smaller than/equal to the pivot, increment i
 - and swap $A[i]$ with $A[j]$, this ensures elements larger than the pivot are moved to the right side of the array

- after going through all elements, swap pivot with the element that's to the right of the last i , in other words between the left and right partitions of the array.
- the new index of the pivot is returned

visual representation of the partition process:



randomized version of quick sort:

- instead of taking the last element as the pivot, the algorithm chooses a random index.
- the main steps are the same as regular quick sort

RANDOMIZED-PARTITION(A, p, r)

```
1  $i = \text{RANDOM}(p, r)$ 
2 exchange  $A[r]$  with  $A[i]$ 
3 return PARTITION( $A, p, r$ )
```

RANDOMIZED-QUICKSORT(A, p, r)

```
1 if  $p < r$ 
2      $q = \text{RANDOMIZED-PARTITION}(A, p, r)$ 
3     RANDOMIZED-QUICKSORT( $A, p, q - 1$ )
4     RANDOMIZED-QUICKSORT( $A, q + 1, r$ )
```

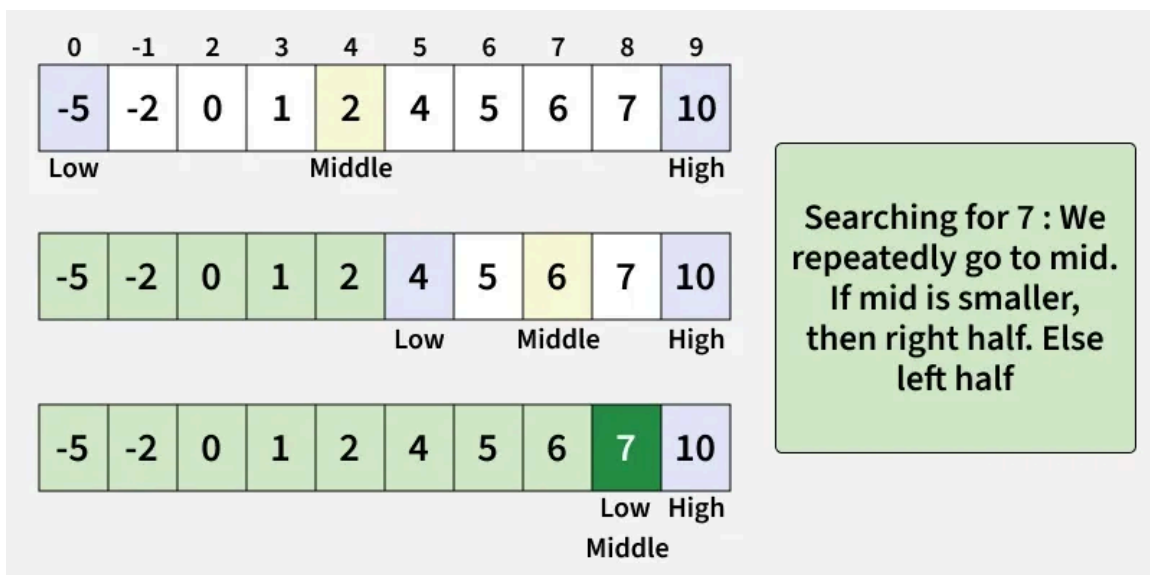
binary search

https://youtu.be/xrMppTpoqdw?si=MRPH8mL4b_ntaG67

binary search is a search algorithm that finds the position of an element within a **sorted** array. very efficient for larger datasets.

two main steps:

1. always start in the middle and check if that element is equal to the target value.
2. if not equal, disregard either the smaller half or greater half, and check the middle value again.



time complexity:

- $O(\log n)$, the larger the dataset, the more efficient binary search becomes compared to other search algorithms

▼ iterative method

<https://youtu.be/C2apEw9pgtw?si=rJWrv3mCJZNI2yhf>

```
BINARY-SEARCH-ITERATIVE(A, n, x) //n=size of array, x = target element
l = 1 //set low as first element
h = n //set high as last element

while l <= h //loop runs as long as low index doesn't cross high index
    mid = (l+h)/2 //set the mid index (round down)

    if A[mid] == x //if mid element is equal to target,
        return mid //return index of mid

    else if A[mid] > x //if mid element is greater than target
        h = mid - 1 //target must be on the left of mid so update h value

    else l = mid + 1 //else x must be on the right of mid so update l

return NIL //otherwise target not in list
```

▼ recursive method (divide and conquer)

<https://youtu.be/uEUXGcc2VXM?si=JmjGCGIDGHxxs7DO>

works the same as the iterative method, but instead of a while loop, call the function again on mid-1 and mid+1.

```

BINARY-SEARCH-RECURSIVE(A, l, h, x)
if l > h //check low index doesnt cross high index
    return NIL
mid = (l+h)/2

if A[mid] == x
    return mid

else if A[mid] > x
    return BINARY-SEARCH-RECURSIVE(A, l, mid-1, x)

else return BINARY-SEARCH-RECURSIVE(A, mid+1, h, x)

```

counting sort

<https://youtu.be/OKd534EWcdk?si=4UnENL2hjnsaIJ-z>

<https://youtu.be/1mh2vilbZMg?si=P79l1ou57EA459nO>

```

COUNTING-SORT( $A, n, k$ )
1  let  $B[1:n]$  and  $C[0:k]$  be new arrays
2  for  $i = 0$  to  $k$ 
3       $C[i] = 0$ 
4  for  $j = 1$  to  $n$ 
5       $C[A[j]] = C[A[j]] + 1$ 
6  //  $C[i]$  now contains the number of elements equal to  $i$ .
7  for  $i = 1$  to  $k$ 
8       $C[i] = C[i] + C[i - 1]$ 
9  //  $C[i]$  now contains the number of elements less than or equal to  $i$ .
10 // Copy  $A$  to  $B$ , starting from the end of  $A$ .
11 for  $j = n$  downto 1
12      $B[C[A[j]]] = A[j]$ 
13      $C[A[j]] = C[A[j]] - 1$  // to handle duplicate values
14 return  $B$ 

```

radix sort

https://youtu.be/XiuSW_mEn7g?si=KcR3MJbL-dhXoq_K

<https://youtu.be/6du1LrLbDpA?si=zTlnON6EWrt5S2aI>

RADIX-SORT(A, n, d)

```
1  for  $i = 1$  to  $d$ 
2      use a stable sort to sort array  $A[1 : n]$  on digit  $i$ 
```

bucket sort

https://youtu.be/JMIYkE8hGJM?si=U95IGeL4N3LHf99_

BUCKET-SORT(A, n)

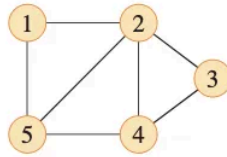
```
1  let  $B[0 : n - 1]$  be a new array
2  for  $i = 0$  to  $n - 1$ 
3      make  $B[i]$  an empty list
4  for  $i = 1$  to  $n$ 
5      insert  $A[i]$  into list  $B[\lfloor n \cdot A[i] \rfloor]$ 
6  for  $i = 0$  to  $n - 1$ 
7      sort list  $B[i]$  with insertion sort
8  concatenate the lists  $B[0], B[1], \dots, B[n - 1]$  together in order
9  return the concatenated lists
```

graph algorithms

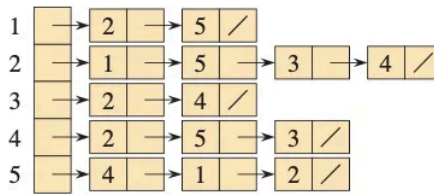
chapters 20-22

graphs

a graph $G = (V, E)$ is a set of vertices(or nodes) V and edges E where each edge (u, v) is a connection between vertices.



(a)



(b)

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

(c)

graphs are represented as either

- a collection of adjacency lists (b)
 - efficient for **sparse** graphs (graphs with much less edges than vertices)
 - looking up an edge takes $O(E)$ time
 - iterating over all edges takes $O(V)$ time
- an adjacency matrix (c)
 - efficient for **dense** graphs (graphs with a lot of edges)
 - looking up an edge takes $O(1)$ time
 - iterating over all edges takes $O(V^2)$ time
- adjacency matrix is fast for looking up edges, but takes up a lot of space.
adjacency list is slow for looking up edges, but fast to iterate over all edges.

different type of graphs:

- **directed graph/undirected graph**: the edges have directions (arrow)/the edges have no directions, so $(u, v) \rightarrow (v, u)$
- **weighted graph/unweighted graph**: the edges have a certain weight/all edges are equal (no weight)
- **cyclic graph/acyclic graph**: a graph with a cycle/a graph with no cycles
- **complete graph**: a graph where there is a unique edge between every pair of vertices
- **tree**: type of graph with 3 key properties:
 1. connected and acyclic

2. removing edge disconnects graph
3. adding edge creates a cycle

the **degree(v)** is the number of edges connected to, ex: in the above pic, degree(1) is 2 because vertex 1 has 2 edges.

- in **directed graphs**, there's a distinction between **in-degree** (incoming edges) and **out-degree** (outgoing edges)
 - a **universal sink v** is a type of vertex in directed graphs with two conditions:
 1. **in-degree**: every other vertex in the graphs has a directed edge pointing to v, the in-degree is $V-1$, where V is total number of vertices.
 2. **out-degree**: there are no edges going out of v, so out-degree is 0.
 - a graph can have at most 1 universal sink.
 - the adjacency matrix representation of a graph with a universal sink v will have:
 - row v: all 0's
 - column v: all 1's except for (v,v) which is 0
 - pseudocode for finding the universal sink in a graph represented as an adjacency matrix, the algorithm passes through each vertex so it takes $O(n)$ or $O(V)$ time.

```

UNIVERSAL-SINK(M, n)//M= adjacency matrix, n=number of nodes/vertices
candidate = 1 //set a 'candidate' for the universal sink

for j=2 to n //iterate through every other vertex j to n-1
    if M[candidate][j] == 1 //if there exists an edge between candidate and j
        candidate = j //candidate cannot be the US, so set j as the new candidate
  
```

```

for i=1 to n //verify the candidate vertex
    if i != candidate
        if M[candidate][i] == 1 OR M[i][candidate] == 0 //
check in- and out-degree
        return 'no universal sink'
    if M[candidate][candidate] == 1:
        return "no universal sink" //US cannot have se
lf loops

return candidate

```

a **path** is a sequence of vertices connected to edges, ex: $1 \rightarrow 2 \rightarrow 3$ is a path.

- the **path length** is the number of edges in a path., ex: $1 \rightarrow 2 \rightarrow 3$ has a path length of 2.
 - a **cycle** is a path that starts and ends at the same vertex, ex: $2 \rightarrow 4 \rightarrow 5 \rightarrow 2$ is a cycle.
- two vertices are **connected** if a path exists between them
 - a graph is **connected** if all vertices are **connected**.
 - a **connected component** is a subset of vertices that is connected, in other words a disconnected graph G made up of vertex sets V1 and V2. V1 is connected and V2 is connected, but V1 and V2 are not connected to each other.

knapsack:

<https://youtu.be/nLmhmb6Nzcm?si=Am0IrxIBEXx4xPou>

BFS (unweighted graphs only)

<https://youtu.be/pcKY4hjDrxk?si=5LZ8CJkYuPipqx9S>

BFS traverses a graph level by level. it uses the **FIFO**(queue) structure to keep track of visited vertices/nodes. BFS is the foundation for solving shortest path

problems

main steps:

- put starting node **s (source)** in the queue and mark as visited.
- while the queue is not empty,
 - dequeue first element from the queue, this becomes the current node
 - visit that node
 - explore neighbours, so for every neighbour of current node that has not been visited yet:
 - mark it as visited
 - enqueue it

pseudocode

- time complexity:
 - adjacency list: $O(V+E)$ with V number of vertices/nodes and E number of edges.
 - BFS performs poorly in dense graphs, the runtime then becomes $O(V^2)$
 - adjacency matrix: $O(V^2)$

```
BFS(G, s)
create empty queue Q
visited = false * G.size //mark every node as not visited
visited[s] = true //start node is visited
ENQUEUE(Q, s) //add start node to queue

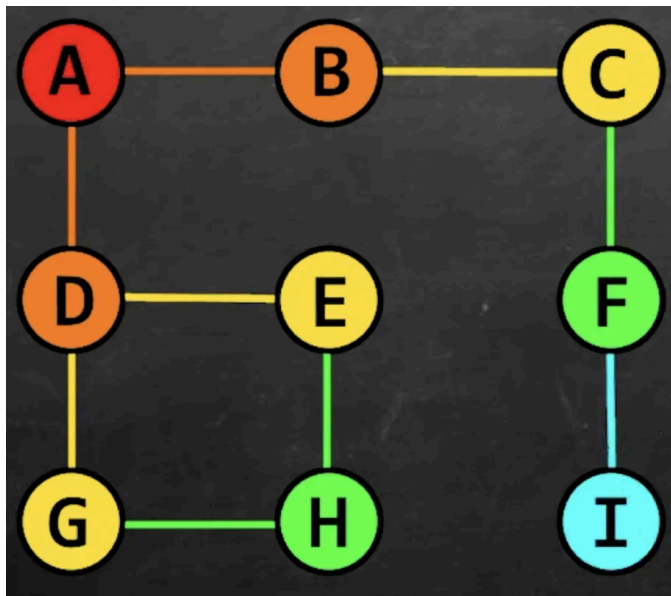
while Q is not empty
    current = DEQUEUE(Q) //get first element in the queue
    visit(current) //process that node

    for neighbour in G.neighbours(current) //visit the neighbours of current node
        if visited[neighbour] == false //if neighbour not yet
```

visited:

```
visited[neighbour] = true    //mark as visited  
ENQUEUE(Q, neighbour) //add to queue to explore next
```

example:



start bfs from node A and explore level by level. the colors represent the order in which the nodes were visited.

1. Q and visited are initialized, Q is empty and visited has all nodes marked as false because none are visited yet.
2. for this traversal, s = A. so A is marked true in visited, and A is then added to Q. right now, A is the only visited node and Q consists only of A.
3. the while loop then starts as long as Q is not empty
 - a. current node to explore is first node from Q (FIFO structure), which right now is A.
 - b. A is then visited, which just means 'look at neighbours of A'
 - c. iterate through all neighbours of A to check if they're true/false in visited. for A, that's nodes B and D. if false, then mark that node as visited to avoid

visiting it again, and enqueue that node in Q.

d. the first iteration of the for-loop marks B as true in visited and enqueues B to Q, so Q now consists of only B (since A was dequeued before to visit)

e. second iteration of the for-loop marks D as true and enqueues D to Q, so Q now consists of [B, Q].

4. now, second iteration of the while loop starts, with the same steps as before. the first element in Q is B, so B is dequeued and then visited, etc. this way the graph is explored level by level.

DFS

<https://youtu.be/pckY4hjDrxk?si=5LZ8CJkYuPipqx9S>

DFS traverses a graph by exploring a path until it reaches a dead end, and then backtracks to last node with unvisited neighbours. it uses a **LIFO** structure to keep track of visited nodes. DFS can be used to:

- find cycles
- find connected components
- find topological ordering of a graph

main steps:

- pick starting node s, and push it on the stack
- visit one of its neighbours and push that on the stack. it doesn't matter which neighbour is visited first.
- then visit neighbour of the last pushed node in the stack.
- after reaching a dead end or already visited node, backtrack to a node that has unvisited neighbours.
 - while backtracking, pop all nodes that were visited from the stack until you reach the node that has unvisited neighbours
- repeat until entire graph is explored/target node is found

pseudocode

- time complexity:
 - adjacency list: $O(V+E)$
 - DFS performs poorly in dense graphs, runtime becomes $O(V^2)$
 - adjacency matrix: $O(V^2)$

```
DFS-RECURSIVE(G, u, visited)
```

```
mark u as visited
```

```
visit(u)
```

```
for each neighbour v in G.neighbours(u) //get the neighbours  
of u
```

```
    if v is not visited //if neighbour is not yet visited
```

```
        DFS-RECURSIVE(G, v, visited) //call the function on t  
his neighbour node
```

```
DFS-ITERATIVE(G, s) //same structure as bfs
```

```
create empty stack S //stack to keep track of visited nodes
```

```
visited = false * G.size
```

```
PUSH(S, s)
```

```
while S is not empty
```

```
    u = POP(S) //get the last pushed element of S
```

```
    if visited[u] == false //if not yet visited, visit this n  
ode
```

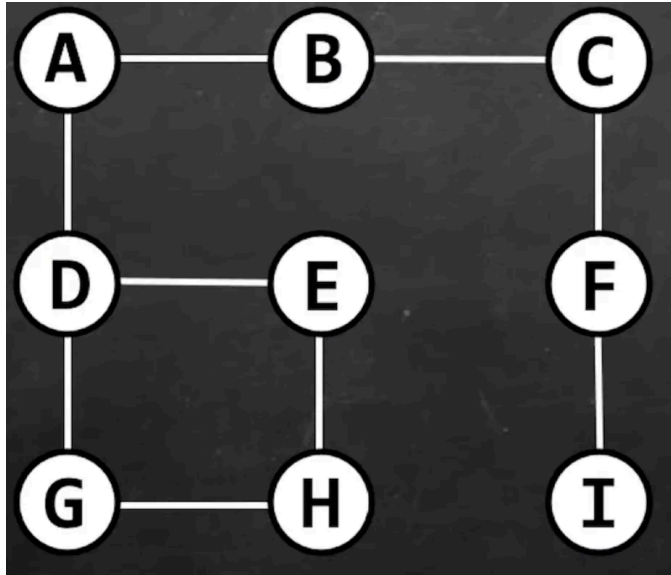
```
        visited[u] = true
```

```
        visit(u)
```

```
        for each neighbour v in G.neighbours(u) //get the nei  
ghbours of u
```

```
            if visited[v] == false //if not yet visited
```

```
                PUSH(S, v) //push to stack so we can backtrac  
k later if needed
```



recursive dfs:

1. $u = A$
2. mark u as visited, so A is marked
3. then get the neighbours of A , which is B and D
4. pick either neighbour, ex B . if B is not yet visited, then call the function again by passing B as the parameter, so inside the for loop you call `DFS-RECURSIVE(G, v, visited)` with $v = B$. the stack now looks like $[A, B]$
 - a. repeat the same steps for B , so mark it as visited.
 - b. then look at the adjacent nodes, which is only C .
 - c. call the function again on C . stack: $[A, B, C]$
 - i. repeat same steps on C
 1. repeat same steps on F . stack: $[A, B, C, F]$
 - a. repeat same steps on I . stack: $[A, B, C, F, I]$
 - b. I results in a dead end, so backtrack to the node with unvisited neighbours.
5. backtrack to A , the only neighbour of A that's not yet visited is D . stack: $[A]$, all the other nodes have been popped when backtracking.

6. repeat until path is exhausted.

iterative dfs:

- it follows the same pseudocode structure as bfs, but in stead of queue we use a stack to keep track of visited nodes. the same logic from recursive dfs applies here, but instead of recursively traversing the graph, we use a while loop to walk through the nodes.

topological sort using DFS

- linear ordering -for Directed Acyclic Graphs(DAG) only- where for each directed edge from A to B, node A appears before node B in the ordering. in other words, B is dependent on/only reachable from A
 - **not** unique, there can be many different topological orderings for a graph.
 - all rooted trees have a topological ordering, since they're directed and acyclic.
 - ex: if you want to take a course H, you must take courses A, B and C before it. so there is a certain ordering when you put this relation in graph form.
- to make a topological ordering, perform DFS on the DAG and only explore unvisited nodes, when backtracking you add the current node (that results in a dead end) to the topological ordering at the end. so the top. ordering is constructed in **reverse** order. example:
 - start DFS at node A in a certain graph, the algorithm finds unvisited neighbour B, so add B to the stack: [A, B].
 - then explore unvisited neighbours of B, stack: [A, B, D]
 - then visit D, this results in a dead end. continue the recursive call of DFS, so backtrack to the last node with unvisited neighbours (by popping nodes from the stack), and put D at the end of the topological ordering.
 - so the topological ordering right now looks like: {_,_,_,_,_,_,_, D}
 - continue DFS until the entire top. ordering is formed.

pseudocode of top sort is just DFS with 1 extra line:

- time complexity: $O(V+E)$ same as DFS

```

TOPOLOGICAL-SORT(G, u, stack)
mark u as visited
//in dfs, the node is visited immediately (pre-order), but not in top sort
for each neighbour v in G.neighbours(u) //get the neighbours of u
    if v is not visited //if neighbour is not yet visited
        DFS-RECURSIVE(G, v, stack) //call the function on this neighbour node
//the node is pushed to the stack after all neighbours are finished (post-order)
PUSH(stack, u)
//when the recursion is done, get the top. ordering by popping the stack

```

- pre order: nodes are added to the traversal order as soon as they're visited
- post order: nodes are added to the traversal order only if a dead end is hit

spanning trees

<https://youtu.be/4ZIRH0eK-qQ?si=ZhvtIJgS3T4nnvg5>

a minimum spanning tree MST is defined for a connected, undirected, weighted graph. the goal is to find an acyclic subset of edges that connect all vertices in the graph with the **smallest possible weight cost** of those edges. this subset is acyclic and connected, so it automatically forms a tree.

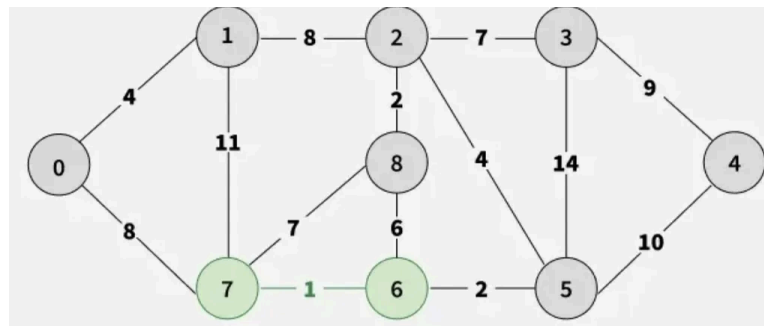
- a graph can have multiple valid MSTs of equal cost
- a MST will always have $V-1$ edges, with V = total amount of vertices. if the MST has V edges, it has cycles (not allowed). and if it has less than $V-1$ edges, the MST is disconnected (not allowed)
- two main ways to get a MST: prims and kruskals algorithms. both are greedy algorithms

kruskal's algorithm

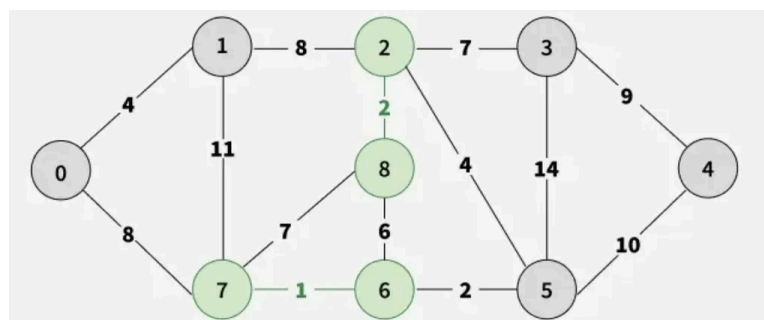
- greedy MST algorithm best for **sparse graphs** (fewer edges), it joins different forests together
- to effectively check for cycles, it uses a **disjoint-set-union (DSU)/union-find** data structure.
- time complexity: $O(E \log V)$

main steps:

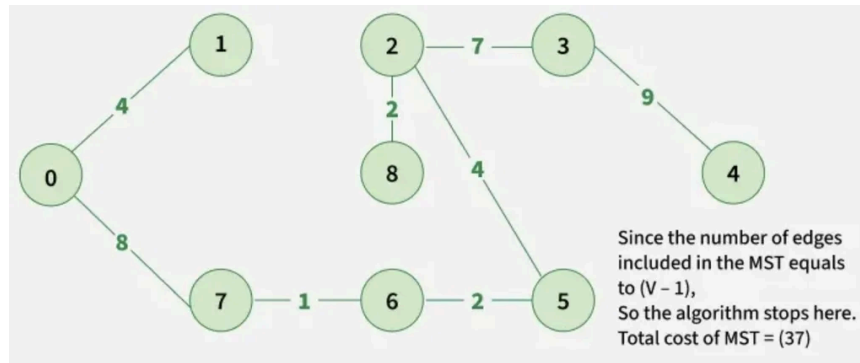
- sort all edges by edge weight in ascending order.
- start from the lowest cost edge and unify the nodes it connects.



- then move on to the next edge with lowest weight, and unify the two nodes it connects



- if selecting a minimum cost edge results in a cycle, don't unify and continue to the next low cost edge.
- continue until all nodes are unified



pseudocode:

```

KRUSKAL(G, s)
A = empty set //initialize empty set
for each v in G.V
    MAKE-SET(v) //make a separate set for each vertex

sort G.E by weight(u, v) //sort edges by weight in ascending
order

for each edge(u, v) in G.E
    if FIND-SET(U) != FIND-SET(V) //check if u and v are in d
    ifferent components
        //in other words it checks if u and v wont form a cycle
        add (u, v) to A //if they are in different componen
        ts, add to A
        UNION(u, v) // and unify u and v

return A

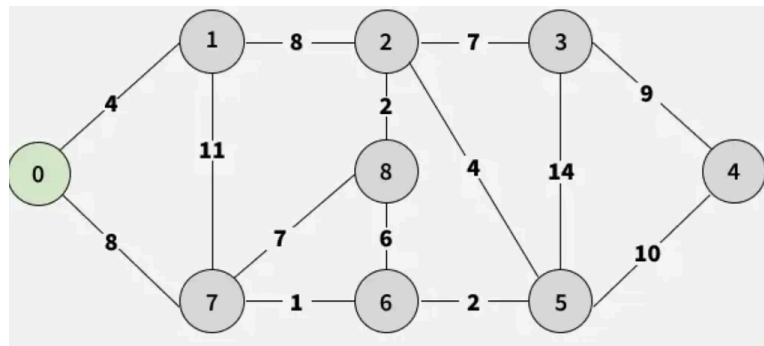
```

prim's algorithm

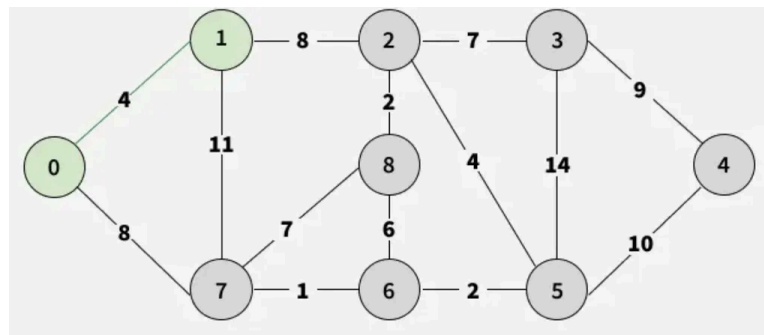
- greedy MST algorithm that works best on **dense graphs** (a lot of edges), it grows a single tree
- not so efficient on disconnected graphs, since the algorithm then must run on each connected component individually.
- time complexity: $O(E \log V)$

main steps:

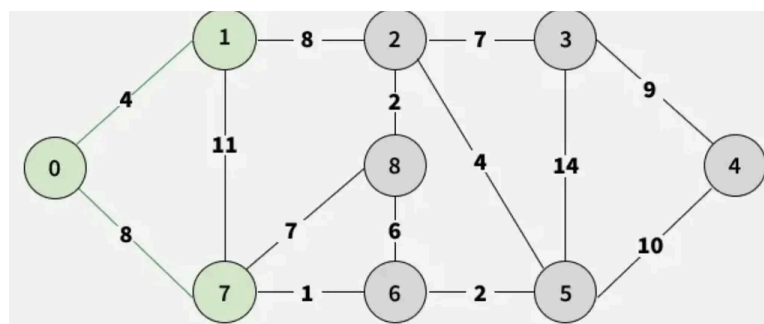
- start from any node in the graph, and examine the edges connected to that node.



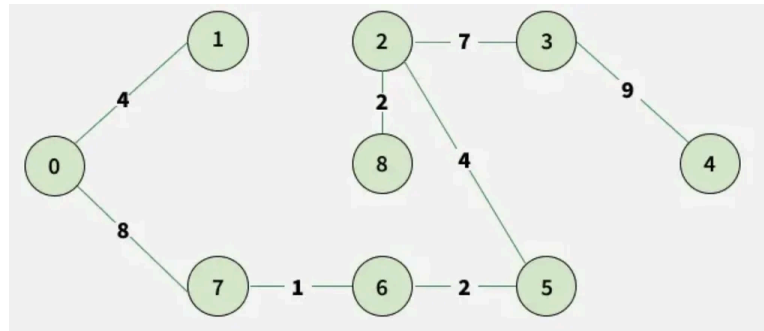
- pick the edge with the smallest weight and add it to the spanning tree with its connecting node. the MST now has two nodes connected with the edge just chosen.



- examine all edges that are connected to the nodes in the MST, and pick the one with the smallest weight and add it to the MST. unlike kruskals algorithm, you must pick an edge that connects to the nodes in the MST.



- repeat until MST is created.



pseudocode:

```

PRIMS(G, s)
dist = array of G.size set to infinity //stores cheapest edge
weight
parent = array of G.size set to Null //structure of MST
in_MST = boolean array set to false //to check if node is in
MST

dist[start_node] = 0 //start node has edge weight of 0
PQ = empty priority queue
PQ.insert(0, start_node)

while PQ is not empty
    u = PQ.extract_min() //add node with minimum edge weight
    to MST

    if in_MST[u] == true //if node is already in MST
        continue //skip and continue
    in_MST[u] = true //otherwise mark it as true

    for each neighbour v in G.adj[u] //check neighbours of u
        if in_MST[v] == false and weight(u, v) < dist[v] //if
v not in MST and edge
            //weight(u, v) is lower than current known way to con
nect to v:

```

```

        dist[v] = weight(u, v) //update dist to weight(u,
v)

        parent[v] = u //set parent of v as u
        PQ.insert(dist[v], v) //add to the priority queue

//the edges(v, parent[v]) will form the MST

```

single source shortest paths

shortest paths and relaxation

- = finding shortest path in a **weighted, directed graph** from a given source vertex/node to every other vertex.
- **relaxation** of an edge(u, v) is checking whether going from vertex u to vertex v improves the shortest path distance found so far for v :

```

RELAX( $u, v, w$ )
if distance[ $u$ ] + cost[ $u, v$ ] < distance[ $v$ ]
    distance[ $v$ ] = distance[ $u$ ] + cost[ $u, v$ ]
    parent[ $v$ ] =  $u$ 

```

- in other words, compare possible path (u, v) to shortest path you currently have and choose the one that's shortest.
- to initialize the source vertex:
 - set distance from s to all other vertices to infinity, since we don't know the optimal distance yet.
 - set parent for all other vertices to null, since there is no path yet to other vertices.
 - distance from s to s is 0.

```

INITIALIZE-SINGLE-SOURCE( $G, s$ )
for each vertex  $v$  in  $G.V$ 

```

```
v.distance = infinity
v.parent = NIL

s.distance = 0
```

bellman-ford algorithm

<https://youtu.be/FtN3BYH2Zes?si=5g4dlQ-wgbbrFsCA>

- not greedy algorithm, works well on graphs with negative edge weights
 - bellman-ford can be used to detect negative cycles in a graph
- at most $V-1$ edges in a shortest path, V is total number of vertices
- time complexity: $O(V^2 + VE)$

```
BELLMAN-FORD(G, w, s) //w=weight, s=source vertex
    INITIALIZE-SINGLE-SOURCE(G, S)

    for i = 1 to |G.V| - 1 //iteratie V-1 times
        for each edge(u, v) in G.E
            RELAX(u, v, w)

    for each edge(u, v) in G.E.
        if v.distance > u.distance + w(u, v)
            return false

    return true
```

- the first for loop is for updating the distances to determine the shortest paths
- the second for loop finds nodes caught in a negative cycle

dijkstra's algorithm

https://youtu.be/XB4MlexjvY0?si=BTTFAYE6V-1_MU3i

- greedy algorithm for shortest paths, but only works on non-negative edge weights.

- dijkstra will get stuck in an infinite loop if there are negative edges, because it will keep finding a shorter path over and over again
- generally faster than bellman-ford. time complexity:
 - $O(E+V\log V)$ improved runtime with fibonnaci heap, wtv that means
 - $O((V+E)\log V)$ average case runtime

```

DIJKSTRA(G, w, s) //w=weight, s=source vertex
  INITIALIZE-SINGLE-SOURCE(G, s)
  S = empty set
  Q = empty min-priority queue

  for each vertex u in G.V
    INSERT(Q, u)

  while Q is not empty
    u = EXTRACT-MIN(Q)
    add u to S

    for each vertex v in G.adj[u]
      RELAX(u, v, w)
      if RELAX decreased v.distance
        DECREASE-KEY(Q, v, v.distance)

```

- empty S to keep track of vertices whose final shortest path from the source have been determined
- min-priority queue Q for easy access to vertices with shortest distance
- while loop:
 - visit vertex with shortest known distance right now and add it to S. in the first iteration that's gonna be the source s, because all other vertices have a distance of infinity right now.

- then look at the neighbours of current visiting vertex to determine which one has shortest distance by calling the relax algorithm.
 - if the relax algorithm finds a shorter path to any of the neighbour vertices v , it will update the currently known distance (which is infinity right now) to the shorter distance. this also means that the vertex v will get a higher priority in the min-priority queue, so the key gets decreased to ensure it's the next first element to get picked from Q .
- then go to next iteration of the while loop and get the next vertex with shortest known distance, which should be a vertex v whose key got decreased in the previous iteration.
 - repeat.

time complexity

time complexity = best/average case, worst case

category	algorithm	time complexity
sorting	insertion sort	$O(n^2)$
	selection sort	$O(n^2)$
	merge sort	$O(n \log n)$
	quick sort	$O(n \log n)$, $O(n^2)$
	bucket sort	$O(n + k)$, $O(n^2)$
heaps	heapify	$O(\log n)$
	heap sort	$O(n \log n)$
	heap insert/delete(extract-__)	$O(\log n)$
	increase/decrease key	$O(\log n)$
graph traversal	BFS	adj.list: $O(V + E)$, $O(V^2)$ adj.matr: $O(V^2)$

category	algorithm	time complexity
	DFS	adj.list: $O(V + E)$, $O(V^2)$ adj.matr: $O(V^2)$
	top. sort	$O(V + E)$
MST	kruskal	$O(E \log V)$
	prims	$O(E \log V)$
shortest path	bellman-ford	$O(V^2 + VE)$
	dijkstra	$O(E + V \log V)$, $O((V+E) \log V)$
BST	insert	$O(\log n)$, $O(n)$
	delete	$O(\log n)$, $O(n)$
	search	$O(\log n)$, $O(n)$
	tree-minimum/maximum	$O(\log n)$
	successor/predecessor	$O(\log n)$, $O(n)$
	pre/in/post - order walk	$O(n)$